

3DGS.zip: A survey on 3D Gaussian Splatting Compression Methods

M. T. Bagdasarian¹  and P. Knoll¹ and Y. Li³  and F. Barthel¹  and A. Hilsmann¹  and P. Eisert^{1,2}  and W. Morgenstern¹ 

¹Fraunhofer HHI, Germany

²Humboldt-Universität zu Berlin, Germany

³Technische Universität Berlin, Germany

Abstract

3D Gaussian Splatting (3DGS) has emerged as a cutting-edge technique for real-time radiance field rendering, offering state-of-the-art performance in terms of both quality and speed. 3DGS models a scene as a collection of three-dimensional Gaussians, with additional attributes optimized to conform to the scene's geometric and visual properties. Despite its advantages in rendering speed and image fidelity, 3DGS is limited by its significant storage and memory demands. These high demands make 3DGS impractical for mobile devices or headsets, reducing its applicability in important areas of computer graphics. To address these challenges and advance the practicality of 3DGS, this state-of-the-art report (STAR) provides a comprehensive and detailed examination of two complementary yet fundamentally distinct strategies: compression and compaction. Compression techniques focus on reducing the file size by encoding Gaussian attributes more efficiently. In contrast, compaction methods directly optimize the scene's structure by optimizing the number of Gaussian primitives. Notably, while methods in both categories aim to maintain or improve quality, each while minimizing its respective attributes—file size for compression and the number of Gaussians for compaction—compaction does not necessarily lead to smaller file sizes; it specifically targets improved efficiency during rendering, making it distinct from compression. We introduce the basic mathematical concepts underlying the analyzed methods, as well as key implementation details and design choices. Our report thoroughly discusses similarities and differences among the methods, as well as their respective advantages and disadvantages. We establish a consistent framework for comparing the surveyed methods based on key performance metrics and datasets. Specifically, since these methods have been developed in parallel and over a short period of time, currently, no comprehensive comparison exists. This survey, for the first time, presents a unified framework to evaluate 3DGS compression techniques. To facilitate the continuous monitoring of emerging methodologies, we maintain a dedicated website that will be regularly updated with new techniques and revisions of existing findings. Overall, this STAR provides an intuitive starting point for researchers interested in exploring the rapidly growing field of 3DGS compression. By comprehensively categorizing and evaluating existing compression and compaction strategies, our work advances the understanding and practical application of 3DGS in computationally constrained environments.

CCS Concepts

• **Information systems** → **Data compression**; • **Computing methodologies** → **Rasterization**; • **General and reference** → **Surveys and overviews**;

1. Introduction

Computer graphics is a constantly evolving field, working towards more realistic and detailed representations of the world. Key milestones in this journey include the introduction of raster graphics in the 1960s, the development of ray tracing in the 1980s to enhance realism, and the adoption of real-time GPU rendering in the 2000s, which transformed gaming and interactive applications.

Realistic 3D world representation remains a crucial and ambitious goal, often considered the "holy grail" for the future of graphics technology. One way to represent the real world is to build it from the ground up with computer graphics methods, while an-

other is to scan and reconstruct real-world scenes. The computer vision community has made significant strides with Neural Radiance Fields (NeRFs) [MST*20], a novel approach to scene representation that leverages neural networks to represent volumetric scenes by predicting the color and density at each point in space.

NeRFs set a new quality standard for rendering, significantly boosting research in 3D rendering and novel view synthesis. However, implicit models like NeRFs come with inherent challenges. They are computationally intensive because the network must be evaluated billions of times for a single image by querying along rays for each pixel. This makes them difficult to manipulate and complicates direct editing of scene elements. Even with advances

such as Instant-NGP [MESK22], which provide much improved training and rendering speeds for radiance fields, the representation remains implicit, limiting its flexibility for practical applications.

3D Gaussian Splatting (3DGS) [KKLD23] has emerged as an explicit alternative to NeRFs, which delivers quality close to the state-of-the-art NeRF approach of Zip-NeRF [BMV*23], while allowing explicit control over scene elements. 3DGS uses Gaussian Splatting to represent a scene by distributing a collection of Gaussian ellipsoids throughout the volume. These Gaussians approximate surfaces and volumes, providing an efficient means of rendering and allowing for more straightforward scene manipulation and editing. 3DGS builds on earlier computer graphics work on splatting techniques [Wes91], such as EWA splatting [ZPVBG02], which laid the foundation for point-based rendering methods.

A key advantage of 3DGS is its ability to handle complex visual effects, such as view-dependent lighting and transparency, which are vital for achieving accurate and visually convincing representations of real-world scenes. As an explicit representation involving Gaussian ellipsoids, 3DGS is editable and can be composed with other representations, such as textured meshes, which are widely used across industries like gaming and film due to their flexibility, editability, and compatibility with existing rendering engines. By supporting features like transparency and view-dependent effects, 3DGS enhances the versatility of 3D scene recording.

Moreover, 3DGS supports real-time rendering on high-end GPUs, creating an optimistic outlook for broader availability of applications on mobile and VR devices in the near future. Its simplicity also contributes to its accessibility: the 3DGS format uses a simple renderer with multiple open implementations and a basic .ply file format [Tur94], which can be read and written by libraries or implementations in many programming languages, making 3DGS accessible from the day it was published. This simplicity has driven rapid adoption across various fields [WYZ*24], making 3DGS an attractive option for both researchers and practitioners.

However, early implementations of 3DGS had significant file size challenges [PKK*24]. Storing all attributes with full precision often resulted in files of several gigabytes, which drove a wave of research focused on compression techniques. Compression of 3DGS files provides several benefits: it allows for faster transmission over slower connections, enables rendering on lower-end devices by reducing memory requirements, and supports the creation of larger, more complex scenes, particularly in video game development. There is thus a strong motivation for compression, leading to an ongoing race to discover the most effective techniques that balance realism and computational efficiency.

The goal of compact scene representation is to achieve an optimal balance between visual realism and efficient storage. While NeRFs perform compression as an integral part of their neural network-based representation, 3DGS requires explicit densification - adding more Gaussian primitives to fill in details - and pruning - removing redundant primitives to optimize memory usage and performance. Despite these challenges, the explicit nature of 3DGS makes it particularly effective for recording real-world scenes, especially with its ability to capture subtle lighting effects and transparency.

This survey provides an overview of the different techniques developed to achieve a compact 3DGS representation, summarizing numerous parallel efforts and evaluating their effectiveness. By examining these methods, the survey guides future research and applications, helping the community identify successful approaches and areas that require further exploration. Many of the techniques used for compressing 3DGS are adaptations of classical methods, tailored for Gaussian splatting, while others introduce entirely new approaches to reduce data size.

The community has been closing the gap in compression efficiency between 3DGS and NeRFs, which may currently serve as an optimal benchmark. By providing a comprehensive overview, we aim to help researchers identify effective strategies and avoid pitfalls, facilitating the development of improved 3DGS compression methods. Establishing a simple and consistent standard for compressed 3DGS will promote broader adoption in the computer graphics community, enhancing its versatility for diverse applications. Such a standard will streamline compatibility and usability, paving the way for 3DGS to achieve the ubiquity and practicality of textured meshes across various use cases.

Scope of this Survey

In this state-of-the-art report, we focus on optimization techniques for 3D Gaussian Splatting (3DGS) representations, aiming to optimize memory usage while preserving visual quality and real-time rendering speed. We focus on compression and compaction methods. We provide a comprehensive comparison of various compression techniques, with quantitative results for the most commonly used datasets summarized in a tabulated format. Specifically, we aim to ensure transparency and create a basis for reproducibility of the included approaches. Additionally, we offer a brief explanation of each pipeline and compare and discuss the main compression and compaction approaches. Rather than covering all existing 3DGS methods and applications, our focus is specifically on techniques to optimize 3DGS representations for size or memory footprint; for a broader overview of 3DGS methods and applications, we refer readers to [WYZ*24, FXZ*24]. While we include common approaches shared between neural radiance field (NeRF) [MST*20] compression and 3DGS compression, we direct readers to [LSW*23, CWHC24] for NeRF-specific compression methods. Furthermore, we acknowledge the inclusion of both peer-reviewed publications from conferences and journals, as well as preprints on arXiv, within our state-of-the-art report, recognizing the rapidly evolving nature of the field of 3D Gaussian splatting.

2. Fundamentals of 3D Gaussian Splatting (3DGS)

3D Gaussian Splatting [KKLD23] introduces a novel approach to real-time radiance field rendering, achieving state-of-the-art performance quality and rendering speed. The technique involves depicting a scene as an ensemble of 3D Gaussian primitives, which are optimized to align with the scene's geometry and visual characteristics. Each 3D Gaussian is characterized by its 59 attributes:

- **Position (μ):** A 3D vector representing the x, y, and z coordinates in world space; the mean of the Gaussian. (3 attributes)

- **Covariance matrix (Σ):** The covariance matrix, representing orientation and size of the Gaussian primitive, can be factorized into the scaling S along the x, y, and z axes (3 attributes) and rotation R , a quaternion, which has 4 components. An exponential activation is used on the scaling parameters. (7 attributes)
- **Opacity (o):** A single scalar value. A sigmoid activation is used to constrain the values to range (0, 1). (1 attribute)
- **Color (c):** View-dependent color is represented by Spherical Harmonics (SH) coefficients. The first degree of spherical harmonics is required to give splats a diffuse color, split into 3 channels (R, G, B). These are denoted $f_dc_{\{0,1,2\}}$ in the .ply point cloud format. Additional view-dependent colors are supported through higher degrees of SH. 3DGS proposes to use degree 3, thus 15 additional coefficients per color channel. (3 rgb x 16 coefficients = 48 attributes)

The training of a static 3D Gaussian Splatting scene utilizes a collection of images as input, in conjunction with calibrated cameras derived from Structure from Motion (SfM), which yield a sparse initial point cloud. For each point within the resulting sparse cloud, a 3D Gaussian distribution ($G(x)$) is initialized. The rendering of 3D Gaussian primitives, also called splatting, is achieved by their projection from a three-dimensional space onto a two-dimensional image plane. Each three-dimensional Gaussian is transformed into a two-dimensional Gaussian (a splat) whose footprint is derived from its covariance matrix and the camera's view transformation parameters. For each pixel, colors are aggregated via alpha-blending, with contributions from each splat blended according to their depth order. Ultimately, the pixel's color, denoted as I , is determined as follows:

$$I(x) = \sum_{i \in N} \alpha_i(x) c_i \prod_{j=1}^{i-1} (1 - \alpha_j(x)), \quad (1)$$

$$\alpha(x) = oG(x), \quad G(x) = e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)}$$

In this context, N denotes the set of depth-sorted splats that overlap at a pixel, c_i is the color and α_i is the contribution of a primitive, that is the product of opacity (o) and Gaussian falloff. This ensures that splats are blended in the appropriate sequence, with those situated closer exerting a more significant impact on the resulting pixel color. Throughout the training or optimization phase, the position, size (as represented by the covariance matrix), opacity, and color of the Gaussian functions are systematically refined to optimally correspond to the input views. Differentiable rendering is used to compute gradients, which allows the adjustment of Gaussian parameters so that the rendered images align with the training images.

Figure 1 shows histograms of the opacity and the first scaling attributes for all 3D Gaussians of the *bicycle* scene (as trained by 3DGS [KKLD23]). Before passing the values to the renderer, their activation functions are applied: sigmoid for the opacity, and exponential for the scale. We can see a peak of very low opacity Gaussians, demonstrating potential candidates to be removed from the scene (see Figure 10). In the scaling histogram, we can see that there are very few large Gaussians, an effect from applying the adaptive density control. Additional histograms for the same scene are shown in the Appendix A.

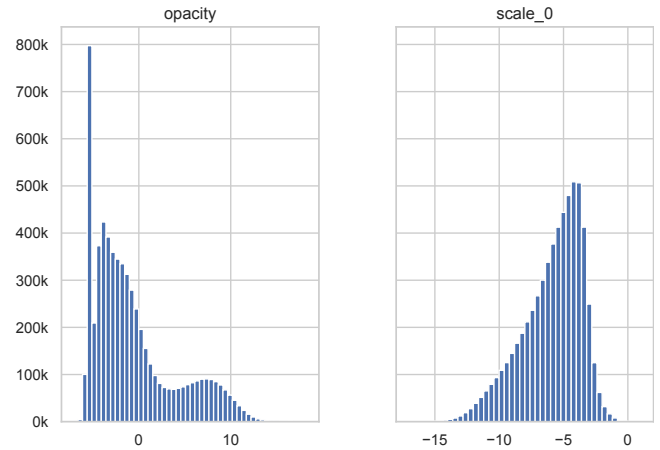


Figure 1: Histograms of the opacity and the first scaling attributes for all 3D Gaussians of the *bicycle* scene (as trained by 3DGS [KKLD23]).

3. Fundamentals of Compression and Compaction for 3DGS

While Gaussian Splatting scenes can efficiently be rendered in real-time, they come with significant file size demands, necessitating efficient compression for managing large scenes. For other media data like images, video, or audio different coding strategies exist to significantly reduce data size. Usually, coding methods are distinguished between lossless and lossy coding. For lossless coding, redundancy is reduced by exploiting the different probabilities of the individual symbols. Such entropy encoding can be achieved, e.g., with Huffman [Huf52] or arithmetic coding [Abr63]. For correlated sources, coding efficiency can be further increased by appropriate prediction or transformation of symbols prior to entropy coding. However, depending on source statistics, lossless coding is often restricted to moderate compression around a factor of 2. Much higher coding efficiency can be expected when tolerating small deviations in the decoded data. Lossy compression targets at removing irrelevance in the data that cannot be perceived by humans but would need additional bits for encoding. Quantization of the predicted or transformed symbols to fewer code words is a standard approach, either for scalar values or an entire vector as in vector quantization (see Sec. 3.1). The goal is then to adjust and shift the quantization error such that it does not become visible.

As an example, Figure 2 shows a correlation heatmap for attributes of all 3D Gaussians of the *bicycle* scene, as provided by 3DGS [KKLD23]. The color channels of the base colors of the splats (f_dc) are nearly perfectly correlated, likely due to the dominance of luminance over chrominance changes in this natural scene. This significant correlation suggests the feasibility of jointly encoding the base color channels. Further correlation exists between the individual scale attributes and also between scale and opacity, which can be exploited in compression. A full correlation map including all spherical harmonics attributes can be found in the Appendix A.

Beyond standard coding techniques, Gaussian Splatting provides numerous opportunities to modify data to support more efficient

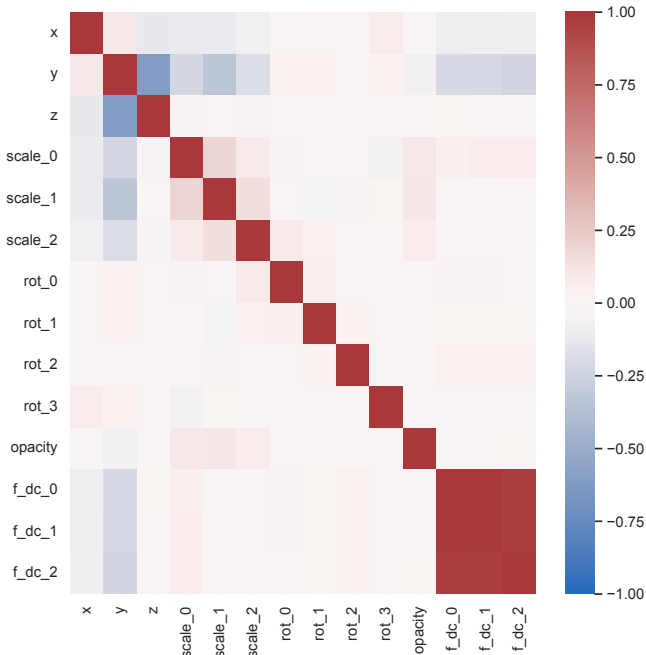


Figure 2: A correlation heatmap for attributes of all 3D Gaussians of the bicycle scene (as trained by 3DGS [KKLD23]).

coding. The reason lies in the different ambiguities within the Gaussian Splatting structure. Different arrangements of splats can lead to the same visual appearance while differing in their compressibility. This can be exploited in considering additional losses during training and scene optimization to limit bit-rate while keeping visual quality. Reparameterizing the 3DGS representation offers another avenue for achieving compression without compromising visual quality. In the following sections, we will first present several concepts for compression and compaction of Gaussian Splatting scenes, while Section 4 will dive into details of individual approaches from the current state of the art.

3.1. Vector Quantization

Vector quantization techniques are central to many compression strategies, with the goal to reduce data complexity by grouping similar data points together and representing them with a shared approximation. Specifically, the original high dimensional dataset is partitioned into clusters, with each cluster being approximated by a representative feature. The process relies on algorithms like K-means [Llo82, Mac67] or LBG [LBG80], iteratively assigning data points to clusters, minimizing the distance between the vectors and their assigned centroids. This leads to the formation of a codebook. Data compression is achieved by replacing each original vector with the index of the closest representative vector in the codebook. Figure 3 illustrates the sequential process starting from unorganized Gaussian attributes, progressing through the clustering, the creation of a codebook, and the final result of Vector Quantization. The quality of the quantized representation heavily depends on the codebook design and the clustering algorithm used.

A well-designed codebook will minimize the error introduced by quantization while maximizing compression efficiency.

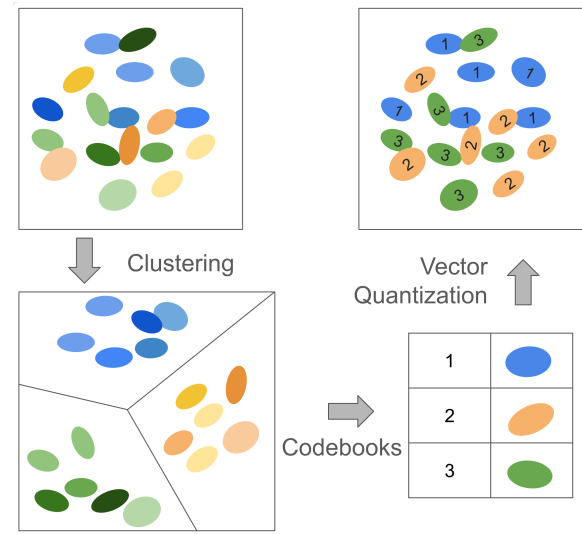


Figure 3: Vector Quantization steps.

The quantization process can either be performed on the entire multi-dimensional vectors simultaneously or individual dimensions of the data space can be treated as separate quantization tasks. The latter case allows for more flexible handling of different data attributes, especially for applications where different properties of the data require different levels of precision and have different distributions and levels of redundancy. In essence, vector quantization algorithms improve a small set of vectors to represent a larger set of vectors under some optimization criterion. This results in significant compression gains, making VQ highly efficient when the data exhibits redundancy or for data-rich applications where a slight loss of precision is acceptable in exchange for dramatic reductions in data size. Hence, these approaches play a crucial role in various domains, from signal processing and image compression to the emerging field of 3D data representation to compress complex datasets while maintaining a close approximation to the original. Especially, it is highly effective for compressing 3D Gaussian splatting (3DGS) data, where attributes like positions, colors, or spherical harmonics are often highly redundant and exhibit natural clustering tendencies.

3.2. Structuring and Dimensionality Reduction

While vector quantization focuses on reducing redundancy by encoding attributes into shared clusters, an alternative perspective integrates structural organization, spatial redundancy exploitation, and dimensionality reduction. Instead of encoding attributes in isolation, methods like octrees, hash-grids, and self-organizing grids reshape and recombine data spatially. These techniques organize scene elements hierarchically or around representative points, enabling efficient reuse of similar properties across regions. By structuring Gaussians to exploit contextual relationships—such as proximity, color, or shape—compression becomes more about the

global arrangement of data, reducing the need to store individual attributes independently. This shift from attribute-focused encoding to structural compression highlights the potential for compact representations through spatial coherence, redundancy management, and transformations into lower-dimensional forms that preserve essential spatial relationships. In this section, we look at different methods that provide either a re-structuring of the data, a dimensionality reduction or factorization, as well as combinations thereof.

3.2.1. Octrees

Octrees are a spatial partitioning technique used in computer graphics to efficiently represent 3D data like point clouds and volumetric models [Mea82]. They have been successfully applied to point cloud compression [SK06]. Furthermore, octrees have demonstrated their effectiveness in speeding up Neural Radiance Field (NeRF) rendering through hierarchical volumetric representation, enabling real-time performance [YLT*21]. Since 3DGS scenes are effectively point clouds with additional attributes, using octrees for 3DGS becomes straightforward. Octrees recursively subdivide 3D space into smaller cubes (see Figure 4), allowing efficient memory allocation by focusing on non-empty regions. In addition, coordinates can be referenced relative to sub-cubes leading to a smaller average bit-length. In 3DGS, octrees help allocate memory only to occupied parts of the scene, skipping over empty regions and, reducing memory usage while preserving detail, making them ideal for large-scale scenes.

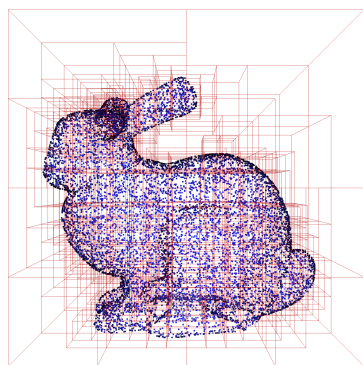


Figure 4: Example of a point cloud with Octree partitioning.

3.2.2. Anchor-based Representations

Anchor points can be used as proxies to predict the properties of associated Gaussian kernels. Unlike octrees, which partition space into a hierarchical grid, anchors group Gaussians by associating them with representative points, allowing efficient compression without strict spatial subdivision [LYX*24]. Anchors are initialized by voxelizing the 3D scene and are assigned position, context features, scaling, and learnable offsets. By deriving Gaussian attributes from these anchors rather than storing them individually, redundancy is reduced, leading to decreased memory usage (Fig. 5). This approach is conceptually related to the codebooks used in

vector quantization presented in Section 3.1, where shared representations efficiently reduce storage requirements while retaining high fidelity.

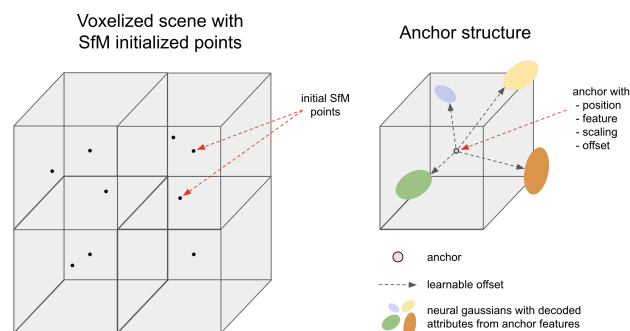


Figure 5: Anchor-based structure. Left: A voxelized scene with SfM initialized points. Right: The center of each voxel becomes an anchor and is associated with position, feature, scaling and offset. From each anchor neural Gaussians are spawned.

3.2.3. Multi-Resolution Hash Grids

Hash-grid assisted context modeling leverages multi-resolution hash grids, inspired by Instant-NGP [MESK22], to efficiently represent spatial relationships in 3D Gaussian Splatting. Instant-NGP's multiresolution hash encoding uses a compact hash table to store trainable feature vectors, which are optimized during training to represent complex spatial details with fewer parameters compared to dense grid encodings. This method enables a trade off between memory, performance, and quality by adjusting parameters such as hash table size, feature vector size, and the number of hash levels. Unlike traditional spatial encodings that explicitly manage collisions, the hash-grid approach uses an MLP to implicitly handle these collisions. This makes this a hybrid explicit/implicit method, and introduces a small computational overhead for the feature decoding. For application in 3DGS, the features stored in the hash table are decoded into attributes for individual primitives. Hash grids are particularly effective in allocating memory to regions of high importance, as demonstrated by Instant-NGP, which achieves small, high-quality radiance fields with minimal parameters. This makes them a very effective tool for 3DGS compression.

3.2.4. Z-order Curves

Z-order curves, or Morton ordering, work by interleaving the bits of the coordinate values from multiple dimensions to create a single one-dimensional index, effectively mapping multidimensional data into a linear sequence while preserving spatial locality. Figure 6 exemplifies how a Z-order curve traverses a regular point grid in 3D space. By ordering the Gaussians according to their positions along a Z-order curve, spatial coherence can be exploited to improve the efficiency of run-length or predictive encoding techniques. This makes Z-order curves useful for spatial indexing and creating a coherent order for splats. In the context of sparse point clouds in 3DGS, Z-order curves are used to order splats based on their positions, which is memory efficient as it avoids allocating storage to empty regions. However, this approach has limitations in terms of

maintaining neighborhood relationships: high-dimensional neighbors are not always close when mapped linearly, which affects the efficiency of operations that rely on true spatial proximity. More sophisticated methods, like Hilbert curves, can sometimes provide better locality for such datasets [CYW22], offering improved efficiency in spatial indexing by ensuring that spatial neighbors are more likely to remain neighbors in the linear mapping.

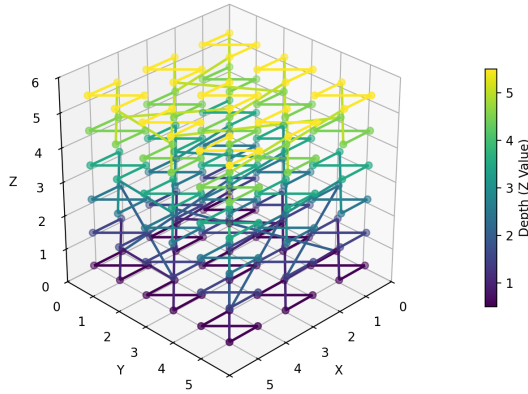


Figure 6: Example of a regular 3D point grid traversed with Z-order curves.

3.2.5. Tri-planes and K-planes

Building on the idea of mapping multidimensional data into lower-dimensional grids, tri-plane factorization instead projects 3D positions onto three orthogonal 2D planes to store and retrieve attributes [CLC*22]. This creates a natural 2D organization, where Gaussians that are spatially close in 3D tend to occupy proximate coordinates on each plane, allowing 2D feature maps to be saved or edited like images while preserving some local neighborhood relationships. Rather than storing each primitive's parameters individually, tri-plane factorization encodes attributes (e.g., color, opacity) as 2D feature images in the xy, xz, and yz planes. This process is visualized in Figure 7. The features are combined via simple operations such as concatenation, summation or multiplication and then decoded through a small MLP. Thus, the method is a hybrid of explicit storage (the planes) and implicit representation (the decoder).

Compared to Z-order curves, which provide a single linear mapping of 3D points to 1D indices, tri-planes rely on three parallel 2D projections for more direct spatial correlation in each axis-aligned view. This makes it straightforward to compress or synthesize the plane images with standard 2D techniques. It also allows for synthesis of the feature planes with generative frameworks such as StyleGAN [Kar19]. Furthermore, extension to higher-dimensional spaces yields K-Planes [FKMW*23], where temporal or additional dimensions can be incorporated by factorizing them into extra 2D planes.

As with other dimensionality reduction methods, key parameters include the overall resolution of each plane (trading off granularity versus memory), how features from the three planes are aggregated, and the complexity of the MLP decoder that maps these aggregated features to the final per-Gaussian attributes. Similar to the Z-order

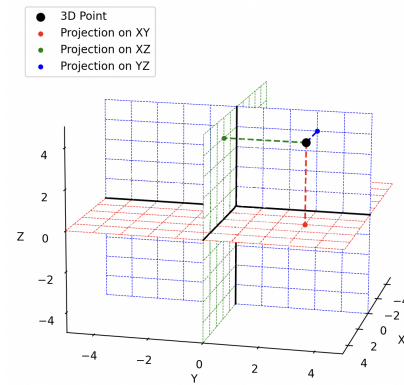


Figure 7: The tri-plane model: A 3D query point is projected onto three orthogonal planes: xy, xz and yz. For 3DGS compression, the query can be the position of the primitive. The planes can store features, which are decoded into the other attributes (e.g. color, opacity) with a small MLP.

approach, the aim is to exploit spatial coherence so that splats sharing similar properties lie close in the 2D feature maps. Because tri-plane factorization scales as $\mathcal{O}(N^2)$ instead of $\mathcal{O}(N^3)$, higher resolutions can be used than in dense voxel grids. This makes tri-plane factorization useful tool for 3DGS compression.

3.2.6. Self-Organizing Gaussians

Another approach to map high-dimensional Gaussian parameters into 2D grids is through self-organization. This method was developed specifically for 3DGS compression [MBHE25]. Figure 8 depicts an example of the primitives of the *Truck* scene from the *Tanks and Temples* dataset reorganized into a 2D grid. The idea is based on the concept of Self-Organizing Maps [Koh90], an unsupervised learning model that projects high-dimensional data onto a lower-dimensional grid, preserving the topological relationships between data points through competitive learning. Similar in motivation to Z-order curves and tri-planes, this 2D representation enables the use of standard image compression techniques, exploiting perceptual redundancies and ensuring local smoothness between neighboring splats. Unlike Z-order curves and tri-planes, which impose a fixed ordering, the Self-Organizing Gaussians representation is optimized for each scene, allowing neighbors in all dimensions to be modeled effectively. By organizing attributes into multiple data layers that share a consistent 2D layout, this technique provides a highly compressible and efficient representation of the original 3D scene.

3.2.7. Region Adaptive Hierarchical Transform (RAHT)

RAHT is a bottom-up transform that reorganizes color attributes along an octree (see also Section 3.2.1 on octrees). At each step, it pairs neighboring voxels according to their weights—the number of underlying voxels—and produces one low-frequency (DC) and one high-frequency coefficient [DQC16]. The DC coefficients propagate upward for further merging, while the high-frequency coefficients are encoded immediately. Because the transform adapts to local density by adjusting the transform matrix to the weights,

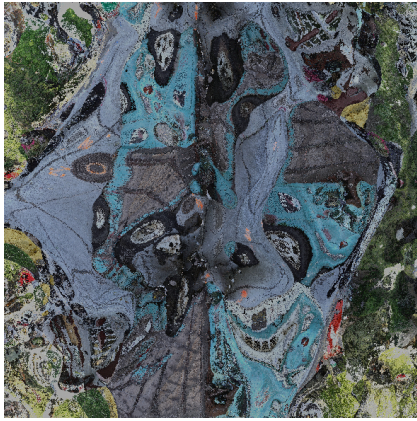


Figure 8: The Gaussian primitives of the Truck scene mapped into a 2D layout using the Self-Organizing Gaussians approach [MBHE25]. An optimal arrangement is learned which preserves local relationships. This learned organization can utilize the available grid space more efficiently than fixed methods like Z-curves tri-planes.

it effectively preserves small-scale details while reducing redundancy. Compared to graph-based transforms that require costly eigen-decompositions, RAHT relies on simple pairwise operations at each level, making it computationally efficient and suitable for real-time compression. After applying the transform, the resulting coefficients are quantized and entropy coded using an arithmetic coder with a Laplacian model, with sub-band parameters encoded with a Run-Length Golomb-Rice (RLGR) coder. This approach achieves compression performance comparable to state-of-the-art methods for point-cloud compression, but at significantly lower complexity. This hierarchical scheme is especially effective for attributes that exhibit strong local correlation, thus it can be adapted to encode attributes like color or density for 3D Gaussian splatting.

3.2.8. Discussion

The methods discussed form a toolkit to achieve compact and efficient 3D Gaussian Splatting, each with distinct advantages and disadvantages. Z-curves, tri-plane factorizations and 2D grid mappings reduce dimensional complexity in a simple and straightforward way. Fixed schemes like Z-order curves provide efficient ordering but may struggle with spatial coherence in sparse datasets. Adaptive techniques, such as anchor-based grouping or learned self-organizing mappings, can mitigate these issues by dynamically adjusting to scene-specific redundancies. Tri-planes may not use the available space efficiently, as empty space in 3D translates to empty patches in the planes.

Self-Organizing Gaussians overcome this by learning a custom, non-linear mapping per scene. Hash-grids use adaptive, multiresolution indexing, which is fast and memory efficient, but is not an invertible mapping. Hash grids and tri-planes both store features, which need to be decoded into attributes with an MLP. While explicit methods allow direct access to Gaussian attributes, this implicit feature storage introduces compactness at the cost of requiring additional decoding.

Octrees adaptively allocate memory only to occupied regions, optimizing resource use for complex scenes, but require advanced attribute compression schemes like RAHT to exploit local correlations for compression. Anchor-based methods group Gaussians around representative points, reducing local redundancy efficiently, similar to vector quantization. While these methods are often considered independently, combining anchor-based representations with structured grids (e.g., hash grids, octrees) can enhance both local and global efficiency. Hybrid approaches allow for flexible spatial partitioning while maintaining fine-grained control over Gaussian placement.

All discussed methods are designed for efficient decoding, making them suitable for real-time applications. However, encoding complexity varies: Self-Organizing Gaussians require a computationally expensive optimization to determine element order, anchor-based methods involve clustering and optimization to define representative points, and feature-based methods like tri-planes and hash grids require a training process to establish features and MLP weights for decoding. This feature decoding adds a small computational overhead at decoding and rendering time.

Memory consumption at decoding and rendering time depends on the representation. For explicit methods, VRAM (GPU memory) usage is directly tied to the number of primitives, meaning that reducing their count through compaction (as discussed in the following sections) directly lowers memory requirements. In contrast, methods like hash grids, tri-planes, and vector quantization can use less memory overall by storing features in compact structures such as feature planes, hash tables, or codebooks. However, their memory usage is constant, meaning that reducing the number of primitives does not further decrease VRAM consumption. Depending on whether a system is memory- or compute-constrained, different trade-offs may be preferable. Large scenes often demand significant VRAM, making compact representations beneficial, while compute-constrained devices, such as VR headsets, may have sufficient memory but benefit from pre-decoding implicit representations into explicit attributes to reduce runtime computation.

3.3. Attribute Pruning

Attribute pruning refers to the selective reduction of attributes associated with each Gaussian, such as spherical harmonics (SH), to optimize memory usage and computational efficiency. In 3DGS, spherical harmonics are often used to encode complex lighting and color information. SH coefficients can become expensive to store and process, especially as the degree of precision increases. For example, adopting SH up to degree 3 for RGB data can result in 48 coefficients (16 per channel), consuming 81% (48 out of 59 attributes) of the storage size required for a single Gaussian.

To address this, attribute pruning techniques dynamically adjust the number of SH coefficients for each Gaussian. Rather than applying a uniform level of SH precision across the entire scene, the degree of SH is tailored based on the specific requirements of each Gaussian. In less complex regions of the scene, some Gaussians only need a single degree of SH (basic RGB color), while more complex regions require higher degrees for more detailed representation.

It is also possible to completely forgo spherical harmonics, and model the view-dependent effects with other approaches, such as NeRF-like MLPs, or classical computer graphics shading [JTL*24]. These approaches may use far fewer attributes than third-degree spherical harmonics, but on the downside require custom, and potentially slower, rendering.

3.4. Compaction

The key tool for achieving compaction in 3D Gaussian Splatting (3DGS) is Adaptive Density Control (ADC). ADC dynamically manages the number of Gaussians during the optimization process, adjusting their density based on scene requirements rather than relying on a fixed number of elements. It adds or removes Gaussians depending on their contribution to the scene, ensuring that only the most essential elements are retained.

By assessing criteria such as gradients, pixel coverage, and saliency maps, ADC intelligently determines whether Gaussians should be cloned, split, or removed. This ensures that additional Gaussians are allocated where they are most needed, such as in high-frequency regions, while redundant or less impactful Gaussians are pruned. Figure 9 exemplifies the processes for (a) Gaussian primitive cloning, (b) Gaussian primitive splitting, and (c) Gaussian primitive pruning. Cloning produces an identical replica of the chosen Gaussian which is then incorporated into the scene. The splitting algorithm substitutes a Gaussian with a set number (default $N = 2$) of child Gaussians. These child Gaussians, after splitting, are situated within the bounds of the removed parent Gaussian. The children's scaling is derived from the parent Gaussian's scaling, reduced by a factor. The remaining attributes are directly inherited from the parent Gaussian.

Figure 10 shows how the number of Gaussians, in this example filtered by opacity, impact the visual quality of the rendered scene. While there is only minimal visual quality loss between 5.6 and 4.1 Million Gaussians, you can already spot changes in the rendering of the bicycle's spokes. This becomes more evident with 3.0 Million Gaussians. With even less Gaussians (1.0 Million) the bicycle reconstruction becomes transparent and the grass reconstruction also suffers.

Operating dynamically during both training and rendering, ADC continuously refines the scene representation. As the scene evolves, ADC ensures efficient Gaussian allocation, leading to a more compact and high-quality model over time.

4. Efficient Strategies for Compression and Compaction in 3D Gaussian Splatting

As 3D Gaussian Splatting (3DGS) evolves as a prominent method for real-time scene rendering, its increasing adoption is challenged by substantial storage and computational requirements. This section addresses two essential optimization strategies: Compression (Sec. 4.1) and Compaction (Sec. 4.2). Compression reduces storage usage by employing methods such as vector quantization, which clusters similar Gaussian attributes to reduce redundancy, and structured representations that organize Gaussians into more compact forms like grids or anchor points. Compaction, on the

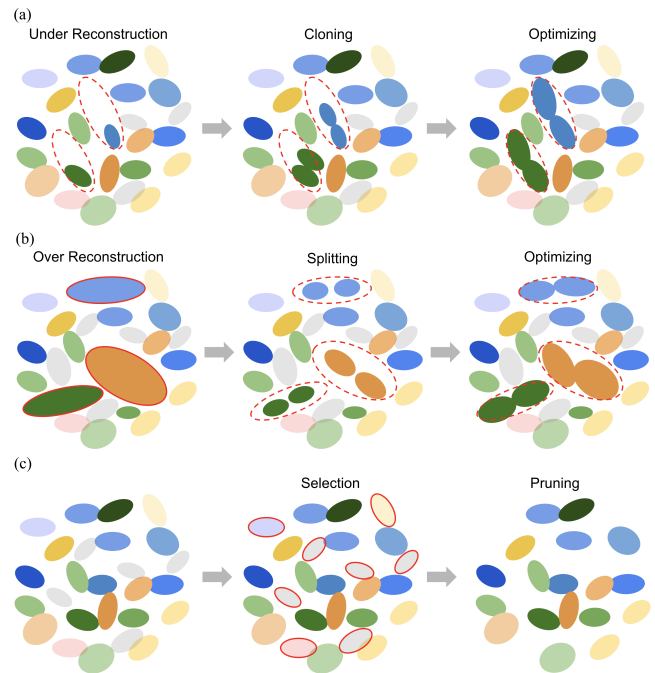


Figure 9: From top to bottom, this figure shows the following processes: (a) Gaussian cloning, (b) Gaussian splitting, and (c) Gaussian pruning.

other hand, focuses on optimizing the number and distribution of Gaussians, ensuring that only the most essential elements are retained while reducing unnecessary data. Figure 11 illustrates how Compression – attribute compression and structured representations – and Compaction intersect. Compaction here includes Gaussian pruning and densification approaches. Together, these strategies enhance the efficiency of 3DGS, making it more practical for a wide range of applications and devices.

4.1. Compression

The principal objective of compression in 3D Gaussian Splatting is to preserve the high fidelity of the original information while substantially reducing data volume. This objective is realized through different strategies that address the primary factors contributing to significant memory usage. 3DGS scenes are composed of a substantial quantity of Gaussians; hence, the most evident method to minimize the memory footprint is to decrease the number of Gaussians utilized (see Sec. 4.2.2). When further examining individual Gaussian kernels, the numerous attributes associated with each Gaussian necessitate the consideration of efficient representations for these attributes. Lastly, Gaussians representing a 3D scene are often not randomly distributed but exhibit spatial relationships and patterns. This inherent structure can be leveraged to organize and compress the Gaussian data more efficiently. Most compression methodologies integrate these various approaches.

In the following sections, we first discuss efficient methodologies for representing Gaussian attributes (Sec. 4.1.1), which is



Figure 10: Render of the Gaussian scene bicycle from the Mip-NeRF 360 dataset with Gaussians pruned based on an opacity criterion.

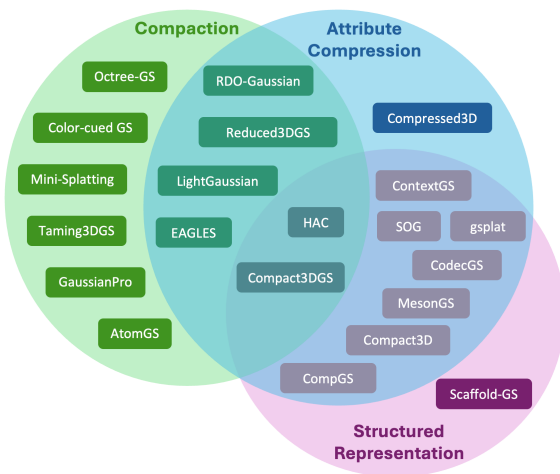


Figure 11: Venn diagram showing how Compression approaches notably Attribute Compression and Structured Representations of Gaussians intersect with Compaction approaches.

then followed by a perspective on the structured representations of Gaussians (Sec. 4.1.2).

4.1.1. Efficient Representation of Gaussian Attributes

3D Gaussian splatting requires significant storage due to the extensive number of Gaussians and their associated attributes. A more efficient representation of these attributes can mitigate storage requirements without appreciable degradation in quality, thereby achieving the objective of compression.

Based on the assumption that many Gaussians share similar attributes, they can be quantized using Vector Quantization (VQ). Most commonly quantized attributes are then stored in codebooks based on K-means as in [NPMKP24, NSW24, PKK*24, LRS*24, FW*24]. In *LightGaussian* [FW*24], the authors employ VQ on Spherical Harmonics in combination with a significance score to omit VQ on Spherical Harmonics with higher significance. Similarly, [NSW24] introduces a sensitivity-parameter which describes the sensitivity of the reconstruction quality to

changes of the Gaussian attributes. This sensitivity measure is then used for sensitivity-aware VQ of Gaussian attributes (e.g. Spherical Harmonics, shape). In [XZT*24], the authors categorize SH coefficients in degrees greater than 0 as unimportant and thus compressible through VQ. Furthermore for key attributes (opacity, scales, Euler angles, and 0-degree SH coefficients), Xie et al. reduce the entropy using RAHT before quantization. *RDO-Gaussian* [WZH*24] use entropy-constrained VQ with codebooks to quantize covariance and color parameters for a more compact representation. *HAC* [CWL*24] uses an anchor structure with associated Gaussians and introduces an Adaptive Quantization Module (AQM) designed to dynamically select quantization steps to facilitate entropy coding of the anchor attributes. The researchers in *EAGLES* [GGS24] employ a methodology for quantizing attributes such as rotation, view-dependent color, and opacity by leveraging a latent vector associated with each attribute. This latent vector is integrated with a multilayer perceptron (MLP) decoder, which facilitates the decoding of latent representations into attribute values. To ensure differentiability during the training process, an additional latent approximation is preserved. A Straight-Through Estimator (STE) is then utilized to round the latent approximation before the propagation of gradients.

As described in Section 2, 3D Gaussians are characterized by 59 attributes, with the majority pertaining to color representation. Of these, the Spherical Harmonics (SH) coefficients encompass 48 attributes across three bands of SH. Notably, 45 out of the 48 SH coefficients are responsible for depicting view-dependent colors. To reduce storage, attribute pruning SH coefficients is a common strategy. In light of this, [PKK*24] introduces an adaptive adjustment mechanism for SH. The authors advocate for the calculation of an average transmittance per pixel and per view. In instances where there is minimal variation per view, this allows for the reduction of SH coefficients to lower bands or their complete removal, thereby significantly diminishing the memory footprint associated with each Gaussian primitive.

Self-Organizing-Gaussians [MBHE25] provide an ablation experiment, showing that training scenes completely without any higher-degree spherical harmonics is possible, still achieving a competitive quality with a much smaller storage size. Building upon the preliminary concept of reducing the number of SH coef-

ficients, *LightGaussian* [FWW*24] proposes a Knowledge Distillation scheme, accompanied by pseudo-view augmentation, to efficiently encapsulate information from higher-order coefficients into a more condensed form.

It should be noted, that whenever a band or degree of spherical harmonics is removed, the higher frequency components of the color representation are also eliminated, which inherently leads to a loss of fine detail in view-dependent effects. This reduction, while beneficial for storage efficiency, inevitably sacrifices some information that would otherwise capture subtle variations in lighting and shading across different viewpoints.

Another straightforward way to reduce memory is using lower bit-depth representations (e.g., 16-bit half-floats instead of 32-bit floats) for attributes that do not require high precision. In *SOG* [MBHE25], the authors clip various attribute ranges, including RGB, opacity, and spherical harmonics, based on percentile thresholds to ensure consistent normalization across models. After clipping, they quantize these attributes by rounding to the nearest value within predefined linear ranges q , with $q = 2^{14}$ for coordinates and $q = 2^6$ for scale, opacity, and rotation and $q = 2^5$ for SH. Fan et al. [FWW*24] quantize selective Spherical Harmonics, position, shape, rotation and opacity attributes. Additionally, packing attributes using a codec like LZ77 as in *MesonGS* [XZT*24] or JPEG XL as in [MBHE25] reduces remaining storage size.

4.1.2. Structured Representation of Gaussians

In the vanilla approach to 3DGS [KKLD23], a final high-fidelity scene usually consists of millions of unordered Gaussians. Besides efficiently representing attributes of single Gaussians by leveraging similarities within attributes as discussed in the previous Sec. 4.1.1, some compression approaches leverage the correlation between neighboring Gaussians and find structured representations, opening new pathways to compression.

One approach for structuring 3D Gaussians within a scene is the Anchor-Based representation, as introduced in *ScaffoldGS* [LYX*24]. From the initial Structure from Motion (SfM) derived point cloud a scene is first voxelized and each voxel center is then treated as an anchor. Each anchor is associated with a local context and learnable offsets, acting as representative point which can spawn new Gaussians in close proximity. Subsequently a neural network predicts the attributes i.e. opacity, color and covariance, of the associated Gaussians based on anchor features and viewing conditions. This approach effectively reduces the overall number of parameters required to represent the scene. *ContextGS* [WLG*24] builds upon the Anchor representation introduced in *ScaffoldGS* [LYX*24] but divides the anchor points into hierarchical levels, from coarse to fine. The anchors are encoded progressively, starting from the coarsest level. The decoded values of the anchors at a coarser level are then used to predict the distribution of nearby anchors at the next finer level, using an MLP. The hierarchical anchor approach efficiently exploits the spatial relationships between anchors. In *HAC* [CWL*24], the Anchor-Based representation is enhanced through the inclusion of a Hash-Grid Assisted Context. The primary concept involves simultaneously learning a structured, compact hash grid for context modeling of anchor attributes. For each anchor, the anchor's position is used

to query the hash grid and retrieve an interpolated hash feature, which subsequently predicts the value distributions of the anchor attributes to aid in entropy coding for a highly efficient representation. While also based on an Anchor-Based representation with anchor primitives and coupled primitives the primitives differ in their structure *CompGS* [LWZ*24]. Anchor primitives serve as reference and contain geometry attributes (location and covariance) and reference embeddings. Coupled primitives only contain residual embeddings to capture deviations. The attributes of coupled primitives are predicted by warping corresponding anchor primitives using affine transforms derived from anchor and coupled primitive embeddings.

In order to leverage the similarities among the color attributes of adjacent Gaussians and to eliminate the need to store attributes for each Gaussian, *Compact3DGS* [LRS*24] proposes the use of a hash-grid followed by a small MLP specifically for view-dependent color attributes. Positions are input into the hash-grid, and the resultant feature along with the view-direction are subsequently provided to the MLP in order to retrieve the color.

An alternative structuring methodology is employed by *SOG* [MBHE25] and also used in *gsplat* [KRS*24], where unstructured Gaussians are mapped onto a structured 2D grid to spatially organize Gaussians with similar attributes closely, enhancing the smoothness of attribute values. This configuration allows for efficient compression using standard image compression techniques. To further improve smoothness and compressibility, a smoothing regularization term can be added during training to promote locally smooth configurations of Gaussians on the grid. An other approach *CodecGS* [LSS*25] applies feature planes for efficient attribute representation and uses a standard video coding technique for feature plane compression. The authors propose a tri-plane architecture to predict Gaussian attributes instead of storing them directly. A progressive training strategy allows to capture coarse geometric information initially and gradually add finer details. The feature planes are optimized by applying a block-wise discrete cosine transform (DCT) allowing the model to leverage the spatial correlations within the feature planes as conventional image signals.

The authors in *Compact3D* [NPMKP24] suggest an approach for organizing the unstructured 3D Gaussians by sorting them according to one of the quantized indices and subsequently storing them employing Run-Length Encoding (RLE).

In *MesonGS* [XZT*24], octrees are employed to achieve compression of the geometrical structure, specifically the 3D positions of the 3D Gaussians.

4.1.3. Quantitative Comparison of Compression Methods

This section presents the comparative analysis of 3DGS compression methods across four datasets: *Tanks and Temples*, *Mip-NeRF 360*, *Deep Blending* and, *Synthetic NeRF*. More details on the datasets are provided in Sec. 5.1. For better comparison Table 1 only includes approaches that mainly focus on compression, we provide a separate Table 2 to compare approaches that mainly focus on compaction (i.e. densification and pruning). Furthermore some approaches were missing the necessary data for quantitative comparison, approaches that we consider worth mentioning nevertheless are included in the survey but not in the tables. As shown in

Table 1, the metrics used to evaluate the performance (see Sec. 5.2) of each compression method are PSNR, SSIM, LPIPS and model size measured in megabytes.

The objectives for compressing 3DGS vary depending on the application, with some requiring minimal model size, while others prioritize a smaller size alongside optimal perceptual quality. As there is no definitive winner that excels across all categories, we introduce a simple rank, reflecting the average rankings of the methods across all available datasets, thereby offering general guidance on the overall performance of the approaches. To determine the compression dataset $rank_s$, the ranks for the quality metrics PSNR, SSIM, and LPIPS are equally weight with the model size. Consequently, each quality metric contributes one-sixth to the overall ranking, while the model size accounts for the remaining half:

$$rank_s = \frac{rank(PSNR)}{6} + \frac{rank(SSIM)}{6} + \frac{rank(LPIPS)}{6} + \frac{rank(SizeMB)}{2}.$$

The overall method ranking is calculated by averaging the dataset ranks across all available datasets. This approach ensures that methods with incomplete data are fairly included in the overall comparison. The $\min$ operator is applied to resolve ties in metric rankings, assigning the lowest rank available to all methods in the group, while subsequent ranks skip the number of tied methods.

The methods evaluated show significant variation in file size, with some achieving high compression rates at the cost of visual quality, while others strike a balance between compression and maintaining higher fidelity. While our proposed ranking puts *ContextGS-lowrate* [WLG*24] on the first place, high variations between datasets and quality metrics show that there is not one winning compression strategy. Depending on the application and goals, different approaches should be considered. When size is the main concern *CodecGS* [LSS*25] stands out for its very small file size (e.g., just 7.8 MB on *Tanks and Temples*), all while maintaining PSNR/SSIM near the top tier. Furthermore, *HAC-lowrate* [CWL*24], *SOG w/o SH* [MBHE25] and *ContextGS_lowrate* [WLG*24] are the best-performing compression methods, achieving file sizes under 10 MB on the *Tanks and Temples* dataset with acceptable levels of visual quality.

When the emphasis is on perceptual quality (LPIPS) *gsplat* [YTtNt] proves to be an excellent choice, alongside *ContextGS_highrate* [WLG*24] and *HAC_highrate* [CWL*24]. It is noteworthy that the original *3DGS-30K* [KKLD23] exhibits the best LPIPS values on both the *Mip-NeRF 360* and the *Deep Blending* dataset, at the cost of being up to 70 times larger in size.

Taken together, the top-ranking methods (*ContextGS* [WLG*24], *HAC* [CWL*24], *CodecGS* [LSS*25], *gsplat* [YTtNt]) achieve a balanced compromise among PSNR, SSIM, LPIPS, and size, with slight differences depending on whether absolute fidelity or smaller storage is the priority. The prevailing conclusion is that methods integrating context modeling (*ContextGS* [WLG*24]) or hierarchical approaches (*HAC* [CWL*24]) tend to dominate in terms of image quality, whereas *CodecGS* and the "lowrate" variants are particularly robust for model compression.

Figure 12 illustrates the trade-offs between model size and performance measured by PSNR for various 3DGS compression methods across different datasets. The curves highlight how smaller file

sizes typically result in lower PSNR, showing the balance between compression efficiency and visual quality. Some approaches have additional data points, which for clarity were not included in the table. Further figures for the performance comparison measured by SSIM and LPIPS as well as the corresponding compaction figures can be found in Appendix B.

4.2. Compaction

Compaction in 3D Gaussian Splatting (3DGS) refers to the optimization of Gaussian kernel distribution in 3D space to accurately represent scene features while maintaining computational efficiency. The initial kernel set often struggles to capture complex details, especially in high-frequency regions or geometrically intricate areas. Compaction in 3D Gaussian Splatting (3DGS) leverages Adaptive Density Control (ADC) to dynamically manage the distribution and density of Gaussians. ADC can be broadly categorized into two key approaches: *Densification* and *Pruning*. This classification is based on the primary objectives of each method: *Densification* techniques focus on selectively adding Gaussians where they are most needed to improve scene fidelity, while *Pruning* techniques focus on removing Gaussians that do not contribute effectively to the scene, avoiding over-reconstruction and inefficiencies. We first introduce densification in Sec. 4.2.1 and pruning in Sec. 4.2.2, outlining how each process contributes to optimizing the representation.

While most of the following methods do not specifically intend to reduce the memory footprint of 3DGS scenes, they implicitly do so, as they improve the quality of the scene without using more Gaussian primitives. Consequently, at similar quality, these methods require less Gaussians and therefore less memory. Note that some methods mentioned in this section are not included in Table 2 due to incomplete evaluations, particularly in the number of Gaussians, which excludes them from the comparison.

4.2.1. Densification

These methods use different criteria to determine where and how to introduce new Gaussians. For instance, the *Color-cued Efficient Densification* method [KLL24] leverages view-independent spherical harmonics coefficients to assess color cues, refining areas where traditional structure-from-motion techniques may struggle to capture fine details. *FreGS* [ZZX*24] addresses over-reconstruction by regularizing frequency discrepancies in rendered images, focusing on the frequency domain. Meanwhile, *Pixel-GS* [ZHL*24] introduces a pixel-aware gradient, targeting under-reconstruction artifacts by incorporating cues from multiple views, thus emphasizing pixel-level information in the densification decision-making process. Similarly, *Revising Densification in Gaussian Splatting (RDGS)* [RBPk24] adopts the criteria that determines whether a Gaussian should be cloned or split during the optimization. Instead of using the accumulated positional gradient of each Gaussian, RDGS uses a structural similarity function to address a loss for each individual Gaussian. If this loss is high enough, the Gaussian will be split during the regular densification intervals. Finally, *Improving Adaptive Density Control (IADC)* [GBHE25] proposes a novel pruning criterion based on alpha blending coefficients, selectively removing Gaussian primitives that minimally contribute to

Table 1: Performance comparison of 3DGS compression methods across four datasets: Tanks and Temples, Mip-NeRF 360, Deep Blending, Synthetic NeRF. The included metrics are PSNR, SSIM, LPIPS and, model size in MB. The best methods in each category are highlighted (*first*, *second*, *third*). The rank represents the average rankings of the methods across all available datasets. The method highlighted in bold: **3DGS-30K** is the original 3DGS method.

Method	Rank	Tanks and Temples				Mip-NeRF 360				Deep Blending				Synthetic NeRF			
		PSNR↑	SSIM↑	LPIPS↓	Size MB↓	PSNR↑	SSIM↑	LPIPS↓	Size MB↓	PSNR↑	SSIM↑	LPIPS↓	Size MB↓	PSNR↑	SSIM↑	LPIPS↓	Size MB↓
ContextGS_lowrate	4.3	24.12	.849	.186	9.9	27.62	.808	.237	13.3	30.09	.907	.265	3.7				
HAC-highrate	4.4	24.40	.853	.177	11.8	27.77	.811	.230	22.9	30.34	.906	.258	6.7	33.71	.968	.034	2.0
CodecGS	4.8	23.63	.841	.192	7.8	27.30	.810	.236	10.3	29.81	.906	.251	9.0				
HAC-lowrate	5.0	24.04	.846	.187	8.5	27.53	.807	.238	16.0	29.98	.902	.269	4.6	33.24	.967	.037	1.2
gsplat-1.00M	5.3	24.03	.857	.163	16.1	27.29	.811	.229	16.0								
ContextGS_highrate	5.8	24.29	.855	.176	12.4	27.75	.811	.231	19.3	30.41	.909	.259	6.9				
Compact3D 32K	9.3	23.44	.838	.198	13.0	27.12	.806	.240	19.0	29.90	.907	.251	13.0				
Compact3D 16K	9.7	23.39	.836	.200	12.0	27.03	.804	.243	18.0	29.90	.906	.252	12.0				
RDO-Gaussian	9.8	23.34	.835	.195	12.0	27.05	.802	.239	23.5	29.63	.902	.252	18.0	33.12	.967	.035	2.3
CompGS	9.8	23.70	.837	.208	10.1	27.26	.803	.239	17.3	29.69	.901	.279	9.2				
Reduced3DGS	10.4	23.57	.840	.188	14.0	27.10	.809	.226	29.0	29.63	.902	.249	18.0				
SOG w/o SH	10.5	23.15	.828	.198	9.3	26.56	.791	.241	16.7	29.12	.892	.270	5.7	31.37	.959	.043	2.0
MesonGS c3	12.0	23.29	.835	.197	17.4	26.99	.797	.246	25.9	29.48	.903	.252	29.0	32.96	.968	.033	3.5
Compressed3D	12.4	23.32	.832	.194	17.3	26.98	.801	.238	28.8	29.38	.898	.253	25.3	32.94	.967	.033	3.7
MesonGS c1	12.6	23.31	.835	.196	18.5	26.99	.796	.247	28.5	29.50	.903	.251	31.1	32.94	.968	.033	3.9
SOG	12.7	23.56	.837	.186	22.8	27.08	.799	.230	40.3	29.26	.894	.268	17.7	33.23	.966	.034	4.1
Compact3DGS+PP	13.3	23.32	.831	.202	20.9	27.03	.797	.247	29.1	29.73	.900	.258	23.8	32.88	.968	.034	2.8
EAGLES	14.4	23.37	.84	.20	29.0	27.23	.81	.24	54.0	29.86	.91	.25	52.0				
Scaffold-GS	14.6	23.96	.853	.177	87.0	27.50	.806	.252	156.0	30.21	.906	.254	66.0				
Compact3DGS	14.9	23.32	.831	.201	39.4	27.08	.798	.247	48.8	29.79	.901	.258	43.2	33.33	.968	.034	5.8
LightGaussian	15.3	23.11	.817	.231	22.0	27.28	.805	.243	42.0					32.72	.965	.037	7.8
3DGS-30K	15.3	23.14	.841	.183	411.0	27.21	.815	.214	734.0	29.41	.903	.243	676.0	33.32			
EAGLES-Small	17.2	23.10	.82	.22	19.0	26.94	.80	.25	47.0	29.92	.90	.25	33.0				

the rendered images. Additionally, IADC introduces an enhanced initialization strategy for the scene extent and employs an exponential gradient threshold, facilitating the generation of more Gaussian primitives early in training and progressively fewer towards convergence.

GaussianPro [CLY*24] employs a different approach by using depth and normal maps to guide the growth and adjustment of Gaussians. It utilizes patch matching [BSFG09] to propagate depth and normal information from neighboring pixels and applies geometric filtering and selection to identify pixels requiring additional Gaussians. *MVG-Splatting* [LYC*24] and *Mini-Splatting* [FW24] also employs a depth map to enforce geometric consistency, but with a more targeted application. Specifically, MVG-Splatting applies the geometric consistency only in the near and far regions of the scene to effectively mitigate under-reconstruction in these critical regions while avoiding the risk of over-reconstruction. Mini-Splatting incorporates depth information after a few optimization iterations. It relies on the initial training process to estimate depth, requiring a few iterations to obtain this information. This approach ensures geometric accuracy while still optimizing the overall scene representation. *Taming 3DGS* [MKVC*24] employs a global scoring approach to guide the addition of Gaussians, ensuring efficient densification. The global score consists of 1) gradient, 2) pixel coverage, 3) per-view saliency, and 4) core attributes like opac-

ity, depth, and scale. By calculating this score that reflects both the scene's structural complexity and visual importance, only the most critical areas are targeted for Gaussian splitting or cloning, resulting in more effective scene representation. *AtomGS* [LXH*24] takes a different approach by employing a density control process similar to the original 3DGS, relying on positional (scaling) gradients. However, it enhances performance through the addition of edge-aware loss, guiding the gradient to better align with the scene's geometry. While it does not explicitly control densification with varying criteria, its refinement in gradient guidance leads to improved density control outcomes.

Some densification approaches adopt a multi-level strategy for greater flexibility. By down sampling or up sampling images, this approach generates multiple Gaussian sets, each finely tuned to different resolution. For instance, *Octree-GS* [RJJ*24] organizes Gaussians according to levels of detail using an octree structure [BLCW23]. It selectively trains Gaussians based on different views, adaptively adjusting the levels for training to ensure optimal rendering based on the observer's perspective.

Lastly, *Markov Chain Monte Carlo (MCMC)* [KRS*24] uses a completely different approach for densifying the 3DGS scene. Instead of cloning and splitting the individual Gaussians, they sample a fixed number of Gaussians from a learned probability distribution.

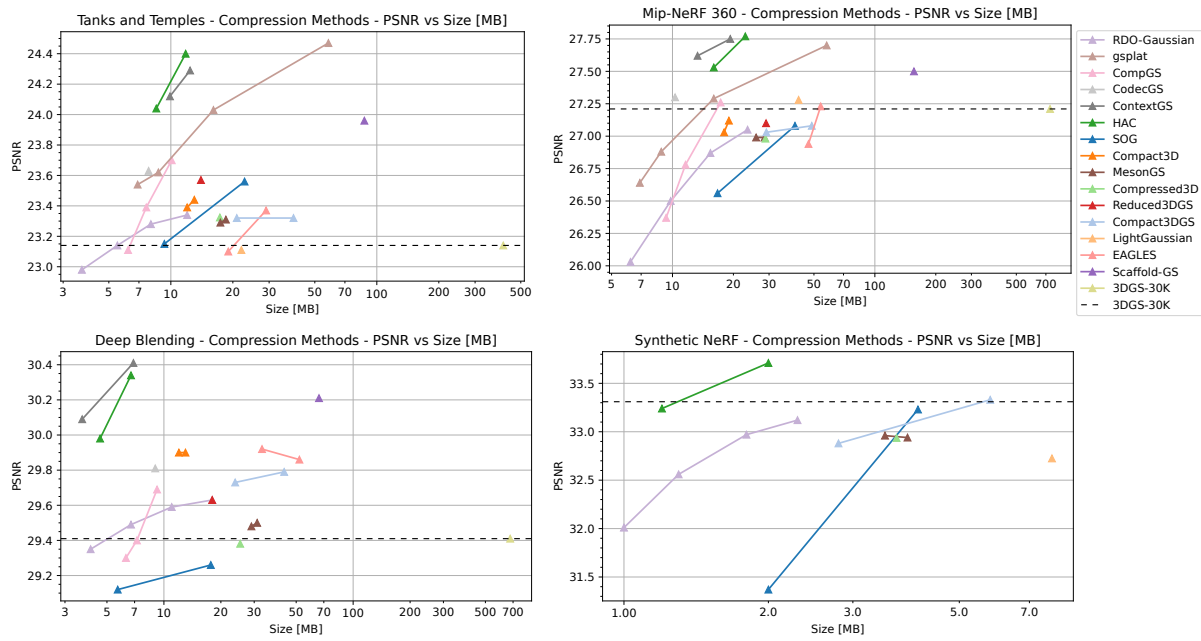


Figure 12: PSNR vs. Model Size (MB) for 3D Gaussian Splatting Compression Methods. The graphs compare different 3DGS compression methods across the Tanks and Temples, Mip-NeRF 360, Deep Blending, and Synthetic NeRF datasets. The x-axis represents the model size (in MB), while the y-axis represents the PSNR, indicating the visual quality.

Specifically, they add a noise term to the position of the Gaussians to incorporate exploration during the training, while they also re-spawn Gaussians that drop below an opacity threshold randomly at Gaussians with high opacity. This way, the method improves the quality of regions with few initial Gaussian primitives and the number of Gaussians can directly be controlled.

4.2.2. Pruning

These techniques focus on identifying and removing redundant Gaussians that contribute minimally to the scene representation, such as those that are excessively large, transparent, or provide overlapping information. *Compact3DGS* [LRS*24], *RDO-Gaussian* [WZH*24], and *HAC* [CWL*24] introduce additional mask parameters to regularize the volume of Gaussians, using binary masks throughout the training process to iteratively eliminate Gaussians based on their contribution, as indicated by the mask values. Both *LightGaussian* [FWW*24] and *EAGLES* [GGS24] use importance-based scoring systems to efficiently eliminate unnecessary Gaussians. *LightGaussian* assigns each Gaussian a global significance score by evaluating its impact on intersected pixels, pruning those with lower significance to enhance efficiency without compromising rendering quality. Similarly, *EAGLES* [GGS24] employs a simplified approach by using a weight composed of transmittance and opacity to represent each Gaussian's importance, ensuring that less significant Gaussians are removed while maintaining overall scene accuracy. Papantonakis et al. [PKK*24] propose a pruning strategy that eliminates Gaussians based on overlap—those with significant overlap with others are considered redundant and removed to reduce overlap without compromising scene accuracy.

SUNDAE [YZJ*24] utilizes a graph-based pruning approach, constructing a graph to capture spatial relationships between Gaussians and applying a band-limited graph filter to selectively down-sample them. To counteract the potential loss of information during this process, a convolutional neural network (CNN) is employed to recover fine details, ensuring a balance between efficient Gaussian placement and the preservation of visual quality.

4.2.3. Quantitative Comparison of Compaction Methods

This section presents a comparative analysis of 3DGS compaction methods across three datasets: *Tanks and Temples*, *Mip-NeRF 360*, and *Deep Blending*. Detailed descriptions of the datasets can be found in Sec. 5.1. As shown in Table 2, the performance of each compression method is evaluated using four metrics: PSNR, SSIM, LPIPS, and the number of Gaussians. Additionally, we compute an overall rank that averages the rankings of the methods across all datasets. To determine the compaction dataset ranks $rank_g$, the quality metrics—PSNR, SSIM, and LPIPS—are given equal weight alongside the number of Gaussians. Specifically, each quality metric contributes one-sixth to the total ranking, while the model size (number of Gaussians, denoted as k Gaussians) accounts for half:

$$rank_g = \frac{rank(PSNR)}{6} + \frac{rank(SSIM)}{6} + \frac{rank(LPIPS)}{6} + \frac{rank(k \text{ Gaussians})}{2}.$$

As for the compression rank calculation the rank for compaction methods is calculated by averaging the rankings over all available datasets, to ensure a fair comparison.

Table 2 highlights that the most efficient compaction methods are those that minimize the number of Gaussians while maintaining reasonable visual quality. *Octree-GS* [RJL*24] and *Mini-Splatting*

[FW24] stand out as the most efficient methods, using the fewest Gaussians while still delivering competitive PSNR values. These methods are ideal for applications requiring tight memory and computation limits. On the other hand, methods like *Taming3DGS (Big)* [MKVC*24] and *GaussianPro* [CLY*24] offer higher quality at the expense of increased Gaussian count, making them more suitable for use cases that prioritize visual fidelity over extreme compaction.

A closer analysis reveals that even the highest-ranked method in terms of efficiency, *Octree-GS*, does not achieve the best performance across all datasets. This suggests that different methods focus on optimizing distinct aspects of the scene. For example, *Octree-GS* excels on datasets like *Deep Blending* and *Tanks and Temples*, where the texture-rich regions are relatively sparse. Its multi-scale strategy enables flexible adjustments of finer details, which allows users to manually refine resolution levels in such regions. In contrast, methods like *Taming3DGS (Big)* perform better in more texture-dense scenes due to their emphasis on higher Gaussian counts and richer detail capture. This underscores the importance of selecting compaction methods based on the specific application needs.

5. Datasets and Comparison Metrics

5.1. Datasets

Performance and quality assessment of 3D Gaussian Splatting algorithms is typically performed on multiple datasets. These datasets provide 3D scenes or objects with various properties, such as varying levels of detail, lighting conditions, and complexities, which allow for comprehensive evaluation of the algorithms.

In our survey, we include *Tanks and Temples* [KPZK17], *Mip-NeRF 360* [BMV*22], *Deep Blending* [HPP*18] as real-world datasets, and *Synthetic NeRF* [MST*21] as a synthetic dataset. Figure 13 shows a sample image from each included scene. From *Tanks and Temples* we include “truck” and “train” two unbounded outdoor scenes which have a centered view point. The *Mip-NeRF 360* dataset also has a centered view point but includes in- and outdoor scenes. The following scenes are included: “bicycle”, “bonsai”, “counter”, “flowers”, “garden”, “kitchen”, “room”, “stump”, “treehill”. From the *Deep Blending* dataset we include “Dr Johnson” and “Playroom” two indoor scenes with a viewpoint directed outward. The synthetic scenes: “chair”, “drums”, “ficus”, “hotdog”, “lego”, “material”, “mic”, “ship” stem from the *Synthetic NeRF* dataset.

These scenes align with those used in the 3D Gaussian Splatting (3DGS) [KKLD23] publication, making them particularly useful for comparing compression methods, as most authors have benchmarked against them. While it would be beneficial to explore larger or more specialized scenes in future work, the lack of accessible data for such comparisons currently limits our scope.

5.2. Comparison Metrics

To evaluate the performance of 3D Gaussian Splatting (3DGS) compression and compaction methods, we rely on a set of well-established metrics that assess both the quality of the rendered scene and the efficiency of the data representation. These metrics

include Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index (SSIM), Learned Perceptual Image Patch Similarity (LPIPS), and model size (in megabytes or number of Gaussians).

- **PSNR** is a widely used metric that quantifies the difference between the original and compressed image in terms of pixel accuracy. Higher PSNR values indicate better fidelity and less distortion.
- **SSIM** measures the perceptual similarity between two images by considering luminance, contrast, and structure. A higher SSIM score represents a closer resemblance between the reference and the rendered scene
- **LPIPS** assesses perceptual quality using a learned model that captures human-like judgments of visual similarity. Lower LPIPS values indicate better perceptual quality.
- **Model Size** is represented either as the file size in megabytes (MB, with 1 MB = 1000^2 Bytes) or the total number of Gaussians in the model. Smaller sizes reflect more efficient compression or compaction.

5.3. Testing Protocols

Initially, our methodology for data compilation for this survey involved parsing various tables from numerous 3DGS compression publications. However, we have then revised our strategy and asked all authors of the publications referenced in this report to provide us with data / results in a standardized format.

In order to ensure consistency and comparability across different approaches, we suggested that authors adhere to the established testing conventions from the original 3DGS project. Specifically, this includes using all 9 scenes from the *Mip-NeRF 360* dataset, incorporating the extra scenes “flowers” and “treehill” (see Sec. 5.1), and only using the “train” and “truck” scenes from *Tanks and Temples*. For image evaluation, full-resolution images should be used up to a maximum side length of 1600px. For larger test images, downscaling is required so that the longest dimension is 1600px, following the standard 3DGS [KKLD23] resizing method, which uses the PIL `.resize()` function with bicubic resampling. For the 3 COLMAP datasets (*Tanks and Temples*, *Deep Blending*, *Mip-NeRF 360*), every 8th image must be selected for testing, specifically those images of index i where $i \bmod 8 \equiv 0$. For the *Synthetic NeRF* dataset, authors should follow the predefined train/evaluation split provided by the dataset.

Table 3 provides a comparative analysis of training and evaluation sizes. The gsplat [YTtNt] methodology was employed to train *Mip-NeRF 360* scenes at three different resolutions: full resolution, 1600 pixels (longest side), and resolutions downsampled by factors of 2 or 4 (longest side < 1600 pixels). The findings indicate that the optimal Peak Signal-to-Noise Ratio (PSNR) results are achieved when the evaluation resolution corresponds precisely to the training resolution. Any deviation, whether by increasing or decreasing the evaluation resolution relative to the training resolution, results in a slight reduction in PSNR. Furthermore, utilizing both higher training and evaluation resolutions enhances PSNR. Additionally, the overall quality is enhanced when employing 1 million Gaussians compared to 360k Gaussians, which aligns with expectations, as rendering scenes with 1 million Gaussians captures more details.

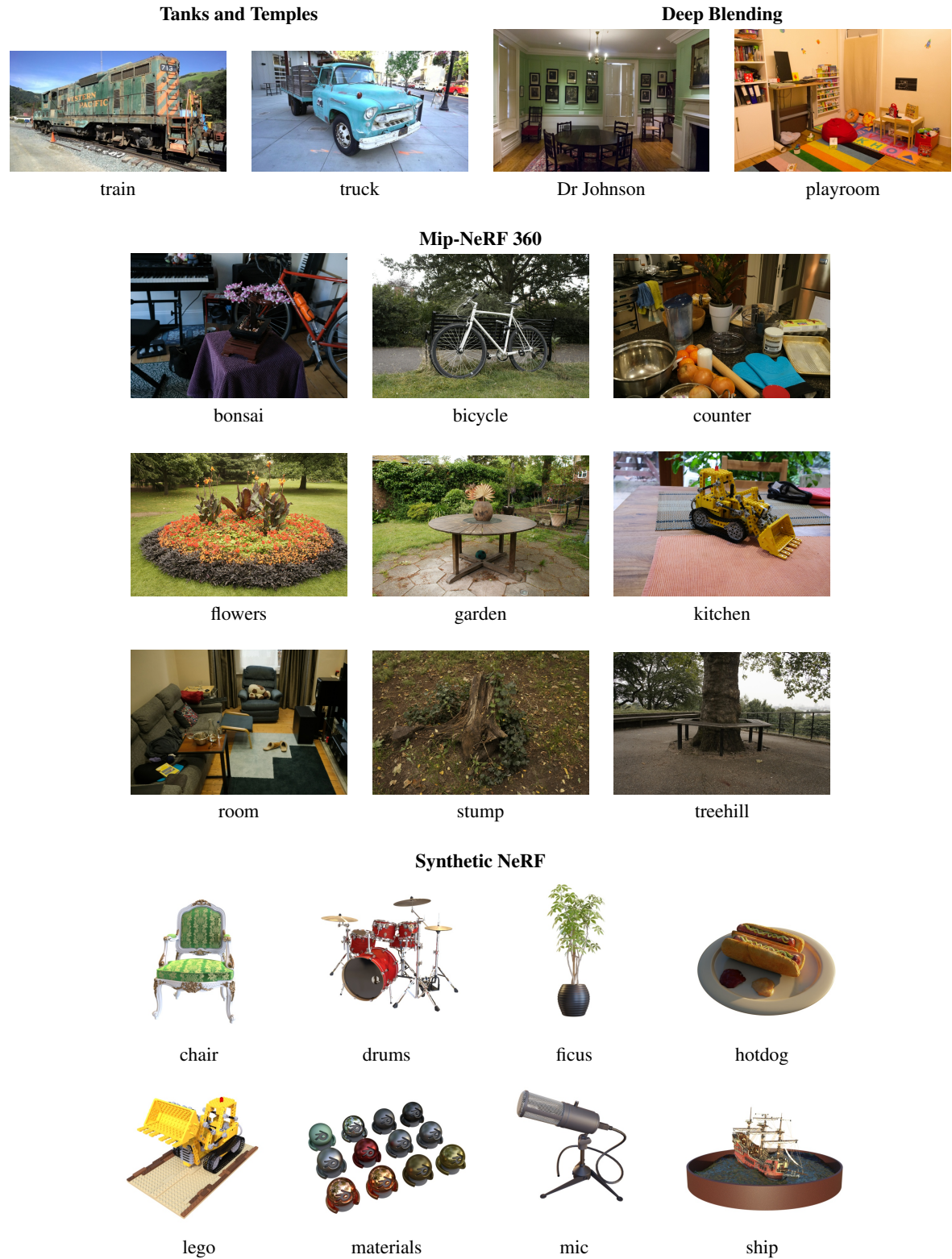


Figure 13: The figure shows a sample image from each scene of the datasets used in evaluation. This selection covers a broad range of scene types, including small to medium-sized natural environments from Tanks and Temples, Deep Blending, and Mip-NeRF 360, alongside highly detailed synthetic scenes with fine textures and reflections provided by Synthetic NeRF. This combination ensures robust evaluation across both real-world and artificial environments.

Table 2: Performance comparison of 3DGS compaction methods across three datasets: Tanks and Temples, Mip-NeRF 360, and Deep Blending. The included metrics are PSNR, SSIM, LPIPS and, number of Gaussians. The best methods in each category are highlighted (first, second, third). The rank represents the average rankings of the methods across all available datasets. The method highlighted in bold: 3DGS-30K is the original 3DGS method.

Method	Rank	Tanks and Temples				Mip-NeRF 360				Deep Blending			
		PSNR↑	SSIM↑	LPIPS↓	k Gauss	PSNR↑	SSIM↑	LPIPS↓	k Gauss	PSNR↑	SSIM↑	LPIPS↓	k Gauss
Octree-GS	2.7	24.68	.866	.153	443	28.05	.819	.217	657	30.49	.912	.241	112
Mini-Splatting	3.4	23.18	.835	.202	200	27.34	.822	.217	490	29.98	.908	.253	350
Taming3DGS	4.8	23.89	.835	.207	290	27.29	.799	.253	630	27.79	.822	.263	270
Taming3DGS (Big)	4.8	24.04	.851	.170	1,840	27.79	.822	.205	3,310	30.14	.907	.235	2,810
AtomGS	4.9	23.70	.849	.166	1,480	27.38	.816	.211	3,140				
GaussianPro	5.0	24.09	.862	.185	1,441	27.43	.813	.219	3,403	29.79	.913	.222	2,582
Color-cued GS	5.5	23.18	.830	.198	370	27.07	.797	.249	646	29.71	.902	.255	644
Mini-Splatting-D	5.7	23.23	.853	.140	4,280	27.51	.831	.176	4,690	29.88	.906	.211	4,630
3DGS-30K	6.6	23.14	.841	.183	1,783	27.21	.815	.214	3,362	29.41	.903	.243	2,975

Table 3: PSNR comparison table for training sizes vs. evaluation sizes. For this table we used gsplat [YtTt] to train Mip-NeRF 360 scenes either in full resolution, with a resolution of 1600 pixels or scaled down with a factor of 2 or 4 (resulting in a resolution less than 1600 pixels, as used in the implementations [BMV*22, KKLD23]). Evaluation is performed using full resolution or 1600 pixels (longest dimension).

#Gaussians →	PSNR (eval. res. full)		PSNR (eval. res. 1600px)	
	360 K	1 M	360 K	1 M
train. res. full	26.38	26.97	26.33	26.67
scaling factor 2/4	25.58	25.84	26.40	27.02
train. res. 1600px	25.82	26.05	26.69	27.33

Consequently, it is advised to carefully select training and evaluation sizes, acknowledging that this decision affects the final PSNR outcome as well as the comparative analysis of different compression approaches.

6. Discussion

The field of 3D Gaussian Splatting (3DGS) has seen rapid advancements, with significant improvements in compression and compaction techniques aimed at overcoming the high storage demands traditionally associated with 3DGS. While the original implementation of 3D Gaussian Splatting [KKLD23] trained on scenes from the Tanks and Temples dataset resulted in a data size exceeding 400 MB, the most effective compression methods achieve more than a 40-fold reduction in size, all the while enhancing visual quality. This state-of-the-art report reveals key insights into these developments, providing a foundation for future optimization and application of 3DGS in real-time rendering.

A crucial finding is that a well-implemented compaction strategy can be an improvement to most methods. Adaptive Density Control (ADC), with densification and pruning, has proven to be effective in significantly enhancing visual quality while reducing the storage footprint. This is vital for applications in resource-constrained environments, such as mobile devices or VR headsets, where storage, memory and processing power are limited

but visual fidelity matters to the user. Structured representations also play a key role in achieving efficient compression. While various compression techniques have been developed independently, their interplay remains an open research area. Understanding the interactions between different methods can lead to more effective strategies, such as vector quantization reducing redundancy by encoding similar Gaussians with shared representations, or structured representations like octrees and hash grids spatially organizing Gaussians to improve compression. Further research into integrating vector quantization within structured representations could lead to more effective encodings.

Techniques that prune Gaussian attributes, such as spherical harmonics (SH) coefficients, must be balanced against compaction strategies like ADC, which reduces the number of Gaussians. Determining the optimal trade-off between reducing attributes and minimizing Gaussian count remains an important area of study. Some compression methods prioritize extreme size reduction through entropy coding and quantization, while others focus on preserving fidelity using hierarchical encoding. Hybrid approaches that dynamically adjust compression levels based on scene complexity could be highly beneficial. Compression techniques must balance reducing file size with maintaining rendering accuracy. Neural feature encoding techniques, such as tri-planes or hash grids, significantly reduce storage needs but require additional computation during rendering. In contrast, explicit representations with precomputed attributes demand more storage but enable real-time rendering with minimal overhead. Lossy compression techniques like pruning or aggressive quantization can introduce artifacts, particularly in view-dependent rendering. Knowledge distillation techniques that encapsulate high-frequency details into lower-order representations offer a promising approach to maintaining perceptual quality while reducing complexity.

Another key challenge is the lack of standard benchmarks for evaluating 3DGS compression methods. Current studies use different datasets, metrics, and experimental conditions, making direct comparisons difficult. In this survey we propose basic guidelines to enable comparison but establishing a unified benchmarking framework with standardized datasets and evaluation protocols would enhance reproducibility and facilitate progress in the field. In addition, methods should be evaluated not only on file size and reconstruction quality but also on real-time performance and energy efficiency, particularly for deployment on mobile and embedded systems.

Finally, the trade-off between compression efficiency and visual quality remains a central challenge. Approaches like HAC-highrate excel in achieving high-quality compression but at the cost of higher computational

overhead. On the other hand, more aggressive compression techniques such as *SOG w/o SH* reduce memory usage significantly but may result in noticeable quality degradation, especially in scenes requiring fine detail. This trade-off highlights the need for flexible solutions that can be tuned based on the specific requirements of the application, whether it's focused on minimizing storage or maximizing visual realism. As the field advances, hybrid approaches that integrate machine learning-driven optimizations may offer new opportunities. Deep learning models could be trained to predict optimal compression strategies based on scene characteristics, dynamically selecting techniques that maximize efficiency while preserving quality.

7. Conclusion and Future Directions

3D Gaussian Splatting (3DGS) has emerged as a powerful alternative to neural radiance fields, offering explicit control over scene elements while achieving high rendering fidelity. However, its practicality remains constrained by storage and computational costs. Scalability, ease of use, and adaptability across different platforms can still be improved. This state-of-the-art-report provides an overview of existing compression and compaction techniques, highlighting key advances and challenges in the field and the importance of continuing to refine and standardize 3D Gaussian Splatting (3DGS) compression and compaction techniques to make them more accessible and widely applicable.

Most current 3DGS compression techniques are designed for static scenes. However, real-world applications increasingly demand dynamic and interactive capabilities. Existing methods rely on pre-trained models, limiting adaptability. Developing real-time adaptive compression strategies that adjust based on scene complexity such as changing objects or lighting conditions over time, or changing hardware constraints would be a valuable research direction. Further advancement could enable real-time simulations and enhance interactive applications such as gaming and virtual reality.

Another improvement could be the creation of multi resolution models with support for level-of-detail (LOD) scaling. Such models would optimize performance by allowing different parts of a scene to be rendered with varying levels of detail, depending on real-time requirements. Octrees or feature grids, provide scalable solutions, but ensuring seamless transitions between different levels of detail without introducing artifacts remains a challenge. Quantization-aware training and the development of shared codebooks across scenes or applications could further improve compression efficiency, allowing for reduced redundancy and more effective memory usage.

Together, these advancements will help expand the applicability of 3DGS to encompass to real-time applications, immersive environments, or large-scale scene representations making 3DGS a more versatile and efficient tool for addressing future challenges in the field of computational graphics.

8. Short Summaries of Included Compression Approaches

In this section, we provide short overviews of the key compression publications surveyed in this report. Each approach offers unique methods to address the challenges of memory and computational efficiency in 3D Gaussian Splatting (3DGS), focusing on aspects such as attribute pruning, vector quantization, and structured representations. By leveraging different strategies, these methods achieve a balance between compression ratio, visual quality, and rendering performance.

The summaries below highlight the primary innovations of each technique, the specific problems they address, and their contributions to advancing 3DGS compression. This section serves as a quick reference for understanding how each method fits into the broader landscape of 3DGS optimization, making it easier to identify the most suitable approach for applications and research needs.

8.1. ContextGS: Compact 3D Gaussian Splatting with Anchor Level Context Model (*ContextGS*)

The authors of *ContextGS* [WLG*24] proposes the first auto-regressive model at the anchor level for 3DGS compression. This work divides anchors into different levels and the anchors that are not coded yet can be predicted based on the already coded ones in all the coarser levels, leading to more accurate modeling and higher coding efficiency. To further improve the efficiency of entropy coding, a low-dimensional quantized feature is introduced as the hyperprior for each anchor, which can be effectively compressed. This work can be applied to both Scaffold-GS and vanilla 3DGS.

8.2. HAC: Hash-grid Assisted Context for 3D Gaussian Splatting Compression (*HAC*)

The paper proposes a Hash-grid Assisted Context (HAC) framework [CWL*24] for compressing 3D Gaussian Splatting (3DGS) models by leveraging the mutual information between attributes of unorganized 3D Gaussians (anchors) and hash grid features. Using Scaffold-GS [LYX*24] as a base model, HAC queries the hash grid by anchor location to predict anchor attribute distributions for efficient entropy coding. The framework introduces an Adaptive Quantization Module (AQM) to dynamically adjust quantization step sizes. Furthermore, this method employs adaptive offset masking with learnable masks to eliminate invalid Gaussians and anchors, by leveraging the pruning strategy introduced by Compact3DGS [LRS*24] and additionally removing anchors if all the attached offsets are pruned.

8.3. Compression of 3D Gaussian Splatting with Optimized Feature Planes and Standard Video Codecs (*CodecGS*)

This method [LSS*25] introduces an effective approach for compressing 3D Gaussian Splatting by employing optimized feature planes and integrating them with standard video codecs. More specifically *CodecGS* introduces progressive tri-planes, where the tri-plane takes 3D point positions x as input and predicts the corresponding attributes for each point. A two-phase training, starting with standard 3DGS [KKLD23] training followed by feature plane training ensures densification is consistent. Progressive training addresses the instability of feature plane training due to sparse 3DGS signals. DCT entropy modeling is employed to transform the feature planes. After training, feature planes are normalized and converted to 16-bit integers, corresponding to the YUV 16-bit format, and are encoded by a standard video codec.

8.4. *gsplat* (*gsplat*)

This approach leverages 3D Gaussian Splatting as Markov Chain Monte Carlo (3DGS-MCMC) [KRS*24], interpreting the training process of positioning and optimizing Gaussians as a sampling procedure rather than minimizing a predefined loss function. Additionally, it incorporates compression techniques derived from the Morgenstern et al. paper [MBHE25], which organizes the parameters of 3DGS in a 2D grid, capitalizing on perceptual redundancies found in natural scenes, thereby significantly reducing storage requirements. Further compression is achieved by clustering spherical harmonics into discrete elements and storing them as FP16 values. This technique is implemented in *gsplat* [YTn], an open-source library designed for CUDA-accelerated differentiable rasterization of 3D Gaussians, equipped with Python bindings.

8.5. Compact3D: Compressing Gaussian Splat Radiance Field Models with Vector Quantization (*Compact3D*)

This approach [NPMKP24] introduces a vector quantization method based on the K-means algorithm to quantize the Gaussian parameters in

3D Gaussian splatting, as many Gaussians may share similar parameters. Only a small codebook is stored along with the index of the code for each Gaussian, resulting in a large reduction in the storage of the learned radiance fields and a reduction of the memory footprint at rendering time. Additionally, the indices are further compressed by sorting the Gaussians based on one of the quantized parameters and storing the indices using a method similar to Run-Length-Encoding (RLE). To reduce the number of Gaussians, this method applies a regularizer to encourage zero opacity before pruning Gaussians with opacity smaller than a threshold.

8.6. CompGS: Efficient 3D Scene Representation via Compressed Gaussian Splatting *CompGS*

The paper CompGS [LWZ*24] proposes a hybrid primitive structure with anchor primitives to predict the attributes of coupled primitives, resulting in compact residual representations. A rate-constrained optimization scheme further enhances compactness by jointly minimizing both rendering distortion and bit rate. The bit rate of both anchor and coupled primitives is modeled by entropy estimation.

8.7. End-to-End Rate-Distortion Optimized 3D Gaussian Representation (*RDO-Gaussian*)

This paper [WZH*24] introduces RDO-Gaussian, an end-to-end Rate-Distortion Optimized 3D Gaussian representation. The authors achieve flexible, continuous rate control by formulating 3D Gaussian representation learning as a joint optimization of rate and distortion. Rate-distortion optimization is realized through dynamic pruning and entropy-constrained vector quantization (ECVQ). Gaussian pruning involves learning a mask to eliminate redundant Gaussians, and adaptive SH pruning assigns varying SH degrees to each Gaussian based on material and illumination needs. The covariance and color attributes are discretized through ECVQ, which performs vector quantization.

8.8. Reducing the Memory Footprint of 3D Gaussian Splatting (*Reduced3DGS*)

This approach [PKK*24] addresses three main issues contributing to large storage sizes in 3D Gaussian Splatting (3DGS). To reduce the number of 3D Gaussian primitives, the authors introduce a scale- and resolution-aware redundant primitive removal method. This extends opacity-based pruning by incorporating a redundancy score to identify regions with many low-impact primitives. To mitigate storage size due to spherical harmonic coefficients, they propose adaptive adjustment of spherical harmonic (SH) bands. This involves evaluating color consistency across views and reducing higher-order SH bands when view-dependent effects are minimal. Additionally, recognizing the limited need for high dynamic range and precision for most primitive attributes, they develop a codebook using K-means clustering and apply 16-bit half-float quantization to the remaining uncompressed floating point values.

8.9. Compact 3D Scene Representation via Self-Organizing Gaussian Grids (*SOG*)

Compressing 3D data is challenging, but many effective solutions exist for compressing 2D data (such as images). The authors propose a new method [MBHE25] to organize 3DGS parameters into a 2D grid, drastically reducing storage requirements without compromising visual quality. This organization exploits perceptual redundancies in natural scenes. They introduce a highly parallel sorting algorithm, PLAS, which arranges Gaussian parameters into a 2D grid, maintaining local neighborhood structure and ensuring smoothness. This solution is particularly innovative because

no existing method efficiently handles a 2D grid with millions of points. During training, a smoothness loss is applied to enforce local smoothness in the 2D grid, enhancing the compressibility of the data. The key insight is that smoothness needs to be enforced during training to enable efficient compression.

8.10. MesonGS: Post-training Compression of 3D Gaussians via Efficient Attribute Transformation (*MesonGS*)

MesonGS [XZT*24] employs universal Gaussian pruning by evaluating the importance of Gaussians through forward propagation, considering both view-dependent and view-independent features. It transforms rotation quaternions into Euler angles to reduce storage requirements and applies region adaptive hierarchical transform (RAHT) to reduce entropy in key attributes. Block quantization is performed on attribute channels by dividing them into multiple blocks and performing quantization for each block individually, using vector quantization for compressing less important attributes. Geometry is compressed using an octree, and all elements are packed with the LZ77 codec. A finetune scheme is implemented post-training to restore quality.

8.11. Compressed 3D Gaussian Splatting for Accelerated Novel View Synthesis (*Compressed3D*)

The authors propose a compressed 3D Gaussian representation [NSW24] consisting of three main steps: 1. sensitivity-aware clustering, where scene parameters are measured according to their contribution to the training images and encoded into compact codebooks via sensitivity-aware vector quantization; 2. quantization-aware fine-tuning, which recovers lost information by fine-tuning parameters at reduced bit-rates using quantization-aware training; and 3. entropy encoding, which exploits spatial coherence through entropy and run-length encoding by linearizing 3D Gaussians along a space-filling curve. Furthermore, a renderer for the compressed scenes utilizing GPU-based sorting and rasterization is proposed, enabling real-time novel view synthesis on low-end devices.

8.12. Compact 3D Gaussian Representation for Radiance Field (*Compact3DGS*)

This approach [LRS*24] introduces a Gaussian volume mask to prune non-essential Gaussians and a compact attribute representation for both view-dependent color and geometric attributes. The volume-based masking strategy combines opacity and scale to selectively remove redundant Gaussians. For color attribute compression, spatial redundancy is exploited by incorporating a grid-based (Instant-NGP) neural field, allowing efficient representation of view-dependent colors without storing attributes per Gaussian. Given the limited variation in scale and rotation, geometric attribute compression employs a compact codebook-based representation to identify and reuse similar geometries across the scene. Additionally, the authors propose quantization and entropy-coding as post-processing steps for further compression.

8.13. EAGLES: Efficient Accelerated 3D Gaussians with Lightweight EncodingS (*EAGLES*)

The authors of this approach [GGS24] observed that in 3DGS, the color and rotation attributes account for over 80% of memory usage; thus, they propose compressing these attributes via a latent quantization framework. Additionally, they quantize the opacity coefficients of the Gaussians, improving optimization and resulting in fewer floaters or visual artifacts in novel view reconstructions. To reduce the number of redundant Gaussians resulting from frequent densification (via cloning and splitting), the approach

employs a pruning stage to identify and remove Gaussians with minimal influence on the full reconstruction. For this, an influence metric is introduced, which considers both opacity and transmittance.

8.14. Scaffold-GS: Structured 3D Gaussians for View-Adaptive Rendering (*Scaffold-GS*)

Scaffold-GS [LYX*24] introduces anchor points that leverage scene structure to guide the distribution of local 3D Gaussians. Attributes like opacity, color, rotation, and scale are dynamically predicted for Gaussians linked to each anchor within the viewing frustum, enabling adaptation to different viewing directions and distances. Initial anchor points are derived by voxelizing the sparse, irregular point cloud from Structure from Motion (SfM), forming a regular grid. Gaussians are spatially quantized using voxels to refine and grow the anchors, with new anchors created at the centers of significant voxels, which are identified by their average gradient over N training steps. Random elimination and opacity-based pruning regulate anchor growth and refinement.

8.15. LightGaussian: Unbounded 3D Gaussian Compression with 15x Reduction and 200+ FPS (*LightGaussian*)

LightGaussian [FWW*24] aims to transform 3D Gaussians to a more efficient and compact form, avoiding the scalability issues that arise from a large number of SfM (Structure from Motion) points for unbounded scenes. Inspired by Network Pruning, the method identifies Gaussians that minimally contribute to scene reconstruction and employs a pruning and recovery process, thereby efficiently reducing redundancy in Gaussian counts while maintaining visual effects. Additionally, LightGaussian utilizes knowledge distillation and pseudo-view augmentation to transfer spherical harmonics coefficients to a lower degree. Furthermore, the authors propose a Gaussian Vector Quantization based on the global significance of Gaussians to quantize all redundant attributes, achieving lower bit-width representations with minimal accuracy losses.

9. Short Summaries of Included Compaction Approaches

Compaction and pruning determine whether to add or remove Gaussians to improve scene accuracy and optimize computational resources. Traditional 3D Gaussian Splatting criteria often fail to capture high-frequency details, causing over- or under-reconstruction. Refined methods better balance efficiency and realism. Compaction identifies where more kernels are needed, particularly in complex or high-frequency areas, while pruning removes superfluous ones that do not add value, enhancing efficiency and rendering fidelity.

9.1. Mini-Splatting: Representing Scenes with a Constrained Number of Gaussians (*Mini-Splatting*)

Mini-Splatting [FW24] enhances Gaussian distribution through Blur Split, which refines Gaussians in blurred regions, and Depth Reinitialization, which repositions Gaussians based on newly generated depth points, calculated from the mid-point of ray intersections with Gaussian ellipsoids, thus avoiding artifacts from alpha blending. For simplification, Intersection Preserving retains Gaussians with the greatest visual impact, while Sampling maintains geometric integrity and rendering quality, reducing complexity.

9.2. Octree-GS: Towards Consistent Real-time Rendering with LOD-Structured 3D Gaussians (*Octree-GS*)

Octree-GS [RJL*24] introduces an octree structure to 3D Gaussian splatting. Starting with a sparse point cloud, an octree is constructed for the

bounded 3D space, where each level corresponds to a set of anchor Gaussians assigned to different levels of detail (LOD). This method selects the necessary LOD based on the observation view, gradually accumulating Gaussians from higher LODs for final rendering. The model is trained using standard image reconstruction and volume regularization losses.

9.3. Taming 3DGS: High-Quality Radiance Fields with Limited Resources (*Taming3DGS*)

Taming 3DGS [MKVC*24] employs a global scoring approach to guide the addition of Gaussians, ensuring efficient densification. Each Gaussian is assigned a score based on four factors: 1) gradient, 2) pixel coverage, 3) per-view saliency, and 4) core attributes like opacity, depth, and scale. Gaussians with the top B scores, where B is the desired number of new Gaussians, are then split or cloned to optimize the scene's representation. By calculating a composite score that reflects both the scene's structural complexity and visual importance, only the most critical areas are targeted for Gaussian splitting or cloning, resulting in more effective scene representation.

9.4. AtomGS: Atomizing Gaussian Splatting for High-Fidelity Radiance Field (*AtomGS*)

AtomGS [LXH*24] prioritizes fine details through Atom Gaussians, which are isotropic and uniformly sized to align closely with the scene's geometry, while large Gaussians are merged to cover smooth surfaces. In addition, Geometry-Guided Optimization uses an Edge-Aware Normal Loss and multi-scale SSIM to maintain geometric accuracy. The Edge-Aware Normal Loss is calculated as the product of the normal map, derived from the pre-optimized 3DGS, and the edge map, which is derived from the gradient magnitude of the ground truth RGB image.

9.5. Color-cued Efficient Densification Method for 3D Gaussian Splatting (*Color-cued GS*)

This method [KLL24] introduces a simple yet effective modification to the densification process in the original 3D Gaussian Splatting (3DGS). It leverages the view-independent (0th) spherical harmonics (SH) coefficient gradient to better assess color cues for densification, while using the 2D position gradient more coarsely to refine areas where structure-from-motion (SfM) struggles to capture fine structures.

9.6. GaussianPro: 3D Gaussian Splatting with Progressive Propagation (*GaussianPro*)

GaussianPro [CLY*24] generates depth and normal maps that guide the growth and adjustment of Gaussians. It employs patch matching to propagate depth and normal information from neighboring pixels to generate new values. Geometric filtering and selection then identify pixels needing additional Gaussians, which are initialized using the propagated information. It also introduces a planar loss to ensure Gaussians match real surfaces more closely. This method enforces consistency between the Gaussian's rendered normal and the propagated normal using L1 and angular loss.

Acknowledgments

This work has partly been funded by German Federal Ministry of Education and Research (BMBF) (MoDL, grant no. 01IS20044), the European Union's Horizon Europe research and innovation program (Luminous, grant no. 101135724), and the German Research Foundation (3DIL, grant no. 502864329). Open Access funding enabled and organized by Projekt DEAL.

References

- [Abr63] ABRAMSON N.: *Information Theory and Coding*. McGraw-Hill, 1963. 3
- [BLCW23] BAI H., LIN Y., CHEN Y., WANG L.: Dynamic plenoc-tree for adaptive sampling refinement in explicit nerf. In *Proceedings of the IEEE/CVF International Conference on Computer Vision* (2023), pp. 8785–8795. 12
- [BMV*22] BARRON J. T., MILDENHALL B., VERBIN D., SRINIVASAN P. P., HEDMAN P.: Mip-nerf 360: Unbounded anti-aliased neural radiance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2022), pp. 5470–5479. URL: <https://jonbarron.info/mipnerf360/>. 14, 16
- [BMV*23] BARRON J. T., MILDENHALL B., VERBIN D., SRINIVASAN P. P., HEDMAN P.: Zip-nerf: Anti-aliased grid-based neural radiance fields. In *Proceedings of the IEEE/CVF International Conference on Computer Vision* (2023), pp. 19697–19705. 2
- [BSFG09] BARNES C., SHECHTMAN E., FINKELSTEIN A., GOLDMAN D. B.: Patchmatch: a randomized correspondence algorithm for structural image editing. *ACM Trans. Graph.* 28, 3 (July 2009). URL: <https://doi.org/10.1145/1531326.1531330>, doi: 10.1145/1531326.1531330. 12
- [CLC*22] CHAN E. R., LIN C. Z., CHAN M. A., NAGANO K., PAN B., DE MELLO S., GALLO O., GUIBAS L. J., TREMBLAY J., KHAMIS S., ET AL.: Efficient geometry-aware 3d generative adversarial networks. In *Proc. IEEE/CVF conference on computer vision and pattern recognition* (2022), pp. 16123–16133. 6
- [CLY*24] CHENG K., LONG X., YANG K., YAO Y., YIN W., MA Y., WANG W., CHEN X.: Gaussianpro: 3d gaussian splatting with progressive propagation. In *Forty-first International Conference on Machine Learning* (2024). 12, 14, 19
- [CWHC24] CHEN Y., WU Q., HARANDI M., CAI J.: How far can we compress instant-ngp-based nerf? In *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2024), pp. 20321–20330. 2
- [CWL*24] CHEN Y., WU Q., LIN W., HARANDI M., CAI J.: Hac: Hash-grid assisted context for 3d gaussian splatting compression. In *European Conference on Computer Vision* (2024), Springer, pp. 422–438. URL: https://yihangchen-ee.github.io/project_hac/. 9, 10, 11, 13, 17
- [CYW22] CHEN J., YU L., WANG W.: Hilbert space filling curve based scan-order for point cloud attribute compression. *IEEE Transactions on Image Processing* 31 (2022), 4609–4621. 6
- [DQC16] DE QUEIROZ R. L., CHOU P. A.: Compression of 3d point clouds using a region-adaptive hierarchical transform. *IEEE Transactions on Image Processing* 25, 8 (2016), 3947–3956. 6
- [FKMW*23] FRIDOVICH-KEIL S., MEANTI G., WARBURG F. R., RECHT B., KANAZAWA A.: K-planes: Explicit radiance fields in space, time, and appearance. In *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2023), pp. 12479–12488. 6
- [FW24] FANG G., WANG B.: Mini-splatting: Representing scenes with a constrained number of gaussians. In *European Conference on Computer Vision* (2024), Springer, pp. 165–181. URL: <https://api.semanticscholar.org/CorpusID:268554047>. 12, 14, 19
- [FWW*24] FAN Z., WANG K., WEN K., ZHU Z., XU D., WANG Z., ET AL.: Lightgaussian: Unbounded 3d gaussian compression with 15x reduction and 200+ fps. *Advances in neural information processing systems* 37 (2024), 140138–140158. URL: <https://lightgaussian.github.io/>. 9, 10, 13, 19
- [FXZ*24] FEI B., XU J., ZHANG R., ZHOU Q., YANG W., HE Y.: 3d gaussian splatting as new era: A survey. *IEEE Transactions on Visualization and Computer Graphics* (2024). 2
- [GBHE25] GRUBERT G., BARTHEL F., HILSMANN A., EISERT P.: Improving adaptive density control for 3D gaussian splatting. In *Proc. 20th Int. Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISAPP)* (Porto, Portugal, 2025), pp. 610–621. 11
- [GGS24] GIRISH S., GUPTA K., SHRIVASTAVA A.: Eagles: Efficient accelerated 3d gaussians with lightweight encodings. In *European Conference on Computer Vision* (2024), Springer, pp. 54–71. URL: <https://efficientgaussian.github.io/>. 9, 13, 18
- [HPP*18] HEDMAN P., PHILIP J., PRICE T., FRAHM J.-M., DRETAKIS G., BROSTOW G.: Deep blending for free-viewpoint image-based rendering. *ACM Transactions on Graphics (ToG)* 37, 6 (2018), 1–15. URL: <http://visual.cs.ucl.ac.uk/pubs/deepblending/>. 14
- [Huf52] HUFFMANN D.: A method for the construction of minimum-redundancy codes. *Proceedings of the IRE* 40, 9 (1952). 3
- [JTL*24] JIANG Y., TU J., LIU Y., GAO X., LONG X., WANG W., MA Y.: Gaussianshader: 3d gaussian splatting with shading functions for reflective surfaces. In *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2024), pp. 5322–5332. 8
- [Kar19] KARRAS T.: A style-based generator architecture for generative adversarial networks. In *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2019). 6
- [KKLD23] KERBL B., KOPANAS G., LEIMKÜHLER T., DRETTAKIS G.: 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics* 42, 4 (July 2023). URL: <https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/>. 2, 3, 4, 10, 11, 14, 16, 17, 22, 23
- [KLL24] KIM S., LEE K., LEE Y.: Color-cued efficient densification method for 3d gaussian splatting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops* (June 2024), pp. 775–783. 11, 19
- [Koh90] KOHONEN T.: The self-organizing map. *Proceedings of the IEEE* 78, 9 (1990), 1464–1480. 6
- [KPZK17] KNAPITSCH A., PARK J., ZHOU Q.-Y., KOLTUN V.: Tanks and temples: Benchmarking large-scale scene reconstruction. *ACM Transactions on Graphics* 36, 4 (2017). URL: <https://www.tanksandtemples.org>. 14
- [KRS*24] KHERADMAND S., REBAIN D., SHARMA G., SUN W., TSENG Y.-C., ISACK H., KAR A., TAGLIASACCHI A., YI K. M.: 3d gaussian splatting as markov chain monte carlo. In *Advances in Neural Information Processing Systems (NeurIPS)* (2024). Spotlight Presentation. 10, 12, 17
- [LBG80] LINDE Y., BUZO A., GRAY R.: An algorithm for vector quantizer design. *IEEE Transactions on Communications* 28, 1 (1980), 84–95. doi:10.1109/TCOM.1980.1094577. 4
- [Llo82] LLOYD S.: Least squares quantization in pcm. *IEEE Transactions on Information Theory* 28, 2 (1982), 129–137. 4
- [LRS*24] LEE J. C., RHO D., SUN X., KO J. H., PARK E.: Compact 3d gaussian representation for radiance field. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2024), pp. 21719–21728. URL: <https://maincold2.github.io/c3dgs/>. 9, 10, 13, 17, 18
- [LSS*25] LEE S., SHU F., SANCHEZ Y., SCHIERL T., HELLGE C.: Compression of 3d gaussian splatting with optimized feature planes and standard video codecs, 2025. URL: <https://arxiv.org/abs/2501.03399>, arXiv:2501.03399. 10, 11, 17
- [LSW*23] LI L., SHEN Z., WANG Z., SHEN L., BO L.: Compressing volumetric radiance fields to 1 mb. In *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2023), pp. 4222–4231. 2
- [LWZ*24] LIU X., WU X., ZHANG P., WANG S., LI Z., KWONG S.: Compgs: Efficient 3d scene representation via compressed gaussian splatting. In *Proceedings of the 32nd ACM International Conference on Multimedia* (2024). 10, 18
- [LXH*24] LIU R., XU R., HU Y., CHEN M., FENG A.: Atomgs: Atomizing gaussian splatting for high-fidelity radiance field. *arXiv preprint arXiv:2405.12369* (2024). 12, 19

- [LYC*24] LI Z., YAO S., CHU Y., GARCIA-FERNANDEZ A. F., YUE Y., LIM E. G., ZHU X.: Mvg-splatting: Multi-view guided gaussian splatting with adaptive quantile-based geometric consistency densification. *arXiv preprint arXiv:2407.11840* (2024). 12
- [LYX*24] LU T., YU M., XU L., XIANGLI Y., WANG L., LIN D., DAI B.: Scaffold-gs: Structured 3d gaussians for view-adaptive rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2024), pp. 20654–20664. URL: <https://city-super.github.io/scaffold-gs/>. 5, 10, 17, 19
- [Mac67] MACQUEEN J.: Some methods for classification and analysis of multivariate observations. In *Proc. of 5-th Berkeley Symposium on Mathematical Statistics and Probability/University of California Press* (1967). 4
- [MBHE25] MORGENSTERN W., BARTHEL F., HILSMANN A., EISERT P.: Compact 3d scene representation via self-organizing gaussian grids. In *Computer Vision – ECCV 2024* (Cham, 2025), Springer Nature Switzerland, pp. 18–34. URL: <https://fraunhoferhhi.github.io/Self-Organizing-Gaussians/>, doi: 10.1007/978-3-031-73013-9_2. 6, 7, 9, 10, 11, 17, 18
- [Mea82] MEAGHER D.: Geometric modeling using octree encoding. *Computer graphics and image processing* 19, 2 (1982), 129–147. 5
- [MESK22] MÜLLER T., EVANS A., SCHIED C., KELLER A.: Instant neural graphics primitives with a multiresolution hash encoding. *ACM transactions on graphics (TOG)* 41, 4 (2022), 1–15. 2, 5
- [MKVC*24] MALLICK AND GOEL, KERBL B., VICENTE CARASCO F., STEINBERGER M., DE LA TORRE F.: Taming 3dgs: High-quality radiance fields with limited resources. In *SIG-GRAPH Asia 2024 Conference Papers* (2024). URL: <https://humansensinglab.github.io/taming-3dgs/>, doi:10.1145/3680528.3687694. 12, 14, 19
- [MST*20] MILDENHALL B., SRINIVASAN P. P., TANCİK M., BARRON J. T., RAMAMOORTHY R., NG R.: Nerf: Representing scenes as neural radiance fields for view synthesis. In *ECCV* (2020). 1, 2
- [MST*21] MILDENHALL B., SRINIVASAN P. P., TANCİK M., BARRON J. T., RAMAMOORTHY R., NG R.: Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM* 65, 1 (2021), 99–106. URL: <https://www.matthewtancik.com/nerf>. 14
- [NPMK24] NAVANEET K., POURAHMADI MEIBODI K., ABASI KOOHPAYEGANI S., PIRSAVASH H.: Compgs: Smaller and faster gaussian splatting with vector quantization. In *European Conference on Computer Vision* (2024), Springer, pp. 330–349. URL: <https://ucdvision.github.io/compact3d/>. 9, 10, 17
- [NSW24] NIEDERMAIR S., STUMPFEGGER J., WESTERMANN R.: Compressed 3d gaussian splatting for accelerated novel view synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2024), pp. 10349–10358. URL: <https://keksboter.github.io/c3dgs/>. 9, 18
- [PKK*24] PAPANTONAKIS P., KOPANAS G., KERBL B., LANVIN A., DRETTAKIS G.: Reducing the memory footprint of 3d gaussian splatting. *Proceedings of the ACM on Computer Graphics and Interactive Techniques* 7, 1 (May 2024), 1–17. URL: https://repo-sam.inria.fr/fungraph/reduced_3dgs/. 2, 9, 13, 18
- [RBP24] ROTA BULÒ S., PORZI L., KONTSCHIEDER P.: Revising densification in gaussian splatting. In *European Conference on Computer Vision* (2024), Springer, pp. 347–362. 11
- [RJT*24] REN K., JIANG L., LU T., YU M., XU L., NI Z., DAI B.: Octree-gs: Towards consistent real-time rendering with lod-structured 3d gaussians. *arXiv preprint arXiv:2403.17898* (2024). 12, 13, 19
- [SK06] SCHNABEL R., KLEIN R.: Octree-based point-cloud compression. In *Proc. Symposium on Point Based Graphics* (2006), vol. 3, pp. 111–121. 5
- [Tur94] TURK G.: The ply polygon file format. *Recuperado de* (1994). 2
- [Wes91] WESTOVER L.: *Splatting: A Parallel, Feed-Forward Volume Rendering Algorithm*. PhD thesis, Univ. of North Carolina at Chapel Hill, 1991. 2
- [WLG*24] WANG Y., LI Z., GUO L., YANG W., KOT A., WEN B.: ContextGS: Compact 3d gaussian splatting with anchor level context model. In *Proc. Conference on Neural Information Processing Systems (NeurIPS)* (2024). 10, 11, 17
- [WYZ*24] WU T., YUAN Y.-J., ZHANG L.-X., YANG J., CAO Y.-P., YAN L.-Q., GAO L.: Recent advances in 3d gaussian splatting. *Computational Visual Media* (2024), 1–30. 2
- [WZH*24] WANG H., ZHU H., HE T., FENG R., DENG J., BIAN J., CHEN Z.: End-to-end rate-distortion optimized 3d gaussian representation. In *European Conference on Computer Vision* (2024), Springer, pp. 76–92. URL: <https://rdogaussian.github.io/>. 9, 13, 18
- [XZT*24] XIE S., ZHANG W., TANG C., BAI Y., LU R., GE S., WANG Z.: Mesongs: Post-training compression of 3d gaussians via efficient attribute transformation. In *European Conference on Computer Vision* (2024), Springer. URL: <https://shuzhaoxie.github.io/mesongs/>. 9, 10, 18
- [YLT*21] YU A., LI R., TANCİK M., LI H., NG R., KANAZAWA A.: PlenOctrees for real-time rendering of neural radiance fields. In *ICCV* (2021). 5
- [YTNt] YE V., TURKULAINEN M., THE NERFSTUDIO TEAM: gsplat. URL: <https://github.com/nerfstudio-project/gsplat>. 11, 14, 16, 17
- [YZJ*24] YANG R., ZHU Z., JIANG Z., YE B., CHEN X., ZHANG Y., CHEN Y., ZHAO J., ZHAO H.: Spectrally pruned gaussian fields with neural compensation. *arXiv preprint arXiv:2405.00676* (2024). 13
- [ZHL*24] ZHANG Z., HU W., LAO Y., HE T., ZHAO H.: Pixel-gs: Density control with pixel-aware gradient for 3d gaussian splatting. In *European Conference on Computer Vision* (2024), Springer, pp. 326–342. 11
- [ZPVBG02] ZWICKER M., PFISTER H., VAN BAAR J., GROSS M.: Ewa splatting. *IEEE Transactions on Visualization and Computer Graphics* 8, 3 (2002), 223–238. 2
- [ZZX*24] ZHANG J., ZHAN F., XU M., LU S., XING E.: Fregs: 3d gaussian splatting with progressive frequency regularization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2024), pp. 21424–21433. 11

Appendix A: Additional figures on attribute statistics.

Appendix B: Additional figures for compression and compaction.

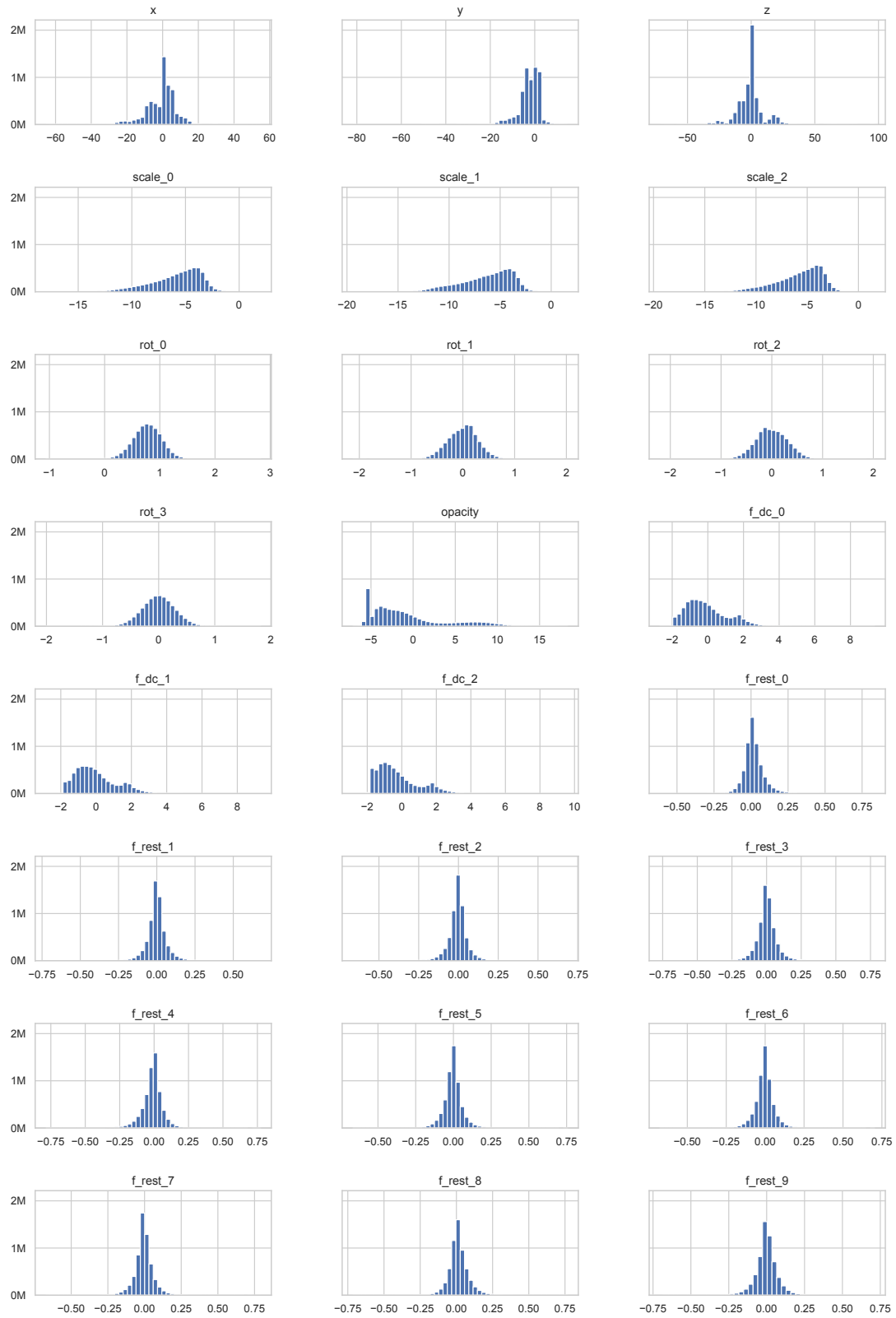


Figure 14: Histograms of attributes for all 3D Gaussians of the bicycle scene, as provided by 3DGS [KKLD23]. Only the first 12 of 48 spherical harmonics attributes are shown for brevity. All attributes are plotted as stored in their files, with their values not activated.

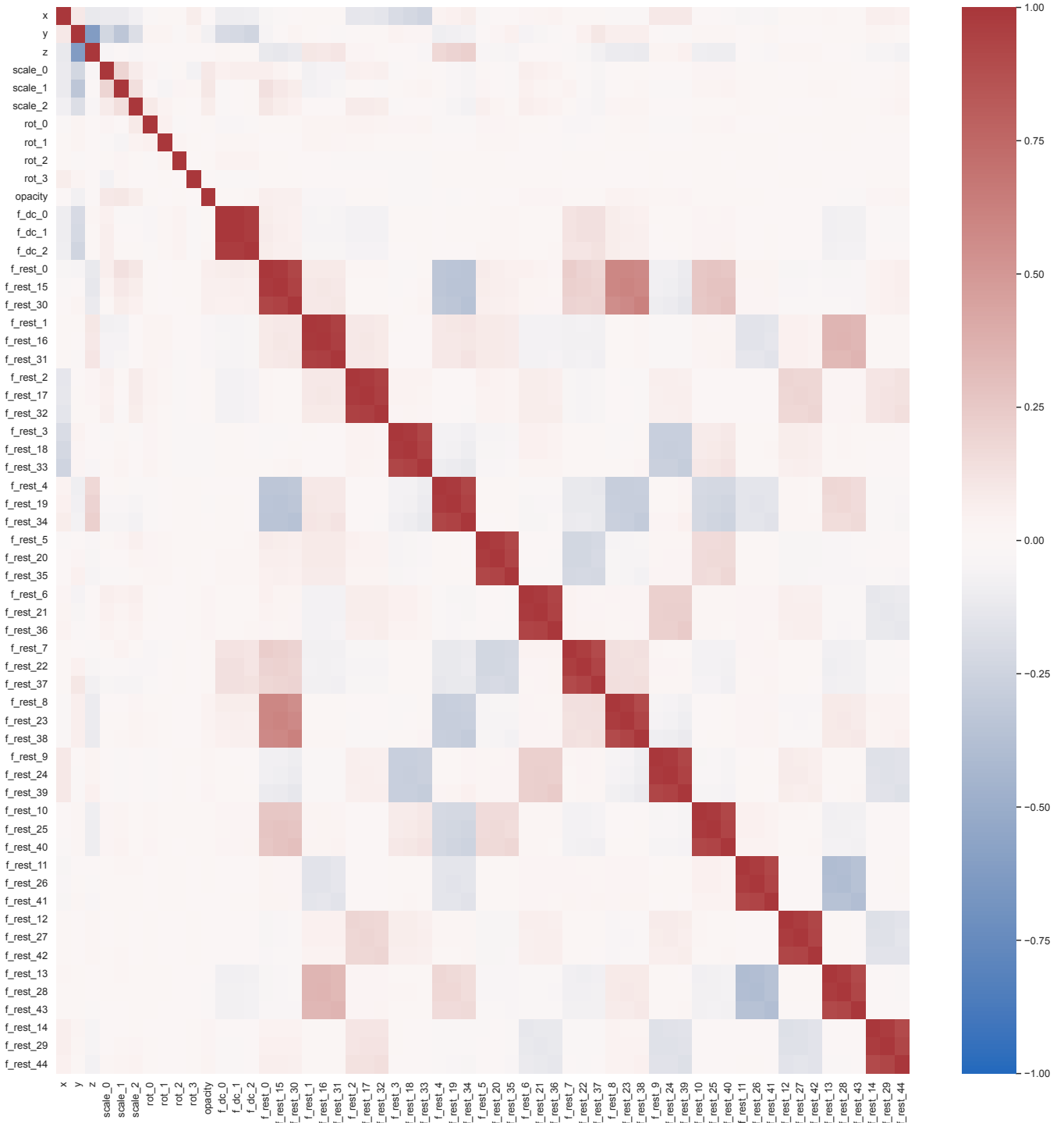


Figure 15: A correlation heatmap for all attributes of all 3D Gaussians of the bicycle scene, as provided by 3DGS [KKLD23]. The color channel correlation seen in the smaller version of this map in Figure 2 is seen repeated in the rest of the attributes of the spherical harmonics (SH), although slowly decreasing in the higher degrees. But there are additional correlations between blocks of these SH attributes, showing potential for compression. The plot also demonstrates the large part of the required space the spherical harmonics attributes take in 3DGS with third-degree SH.

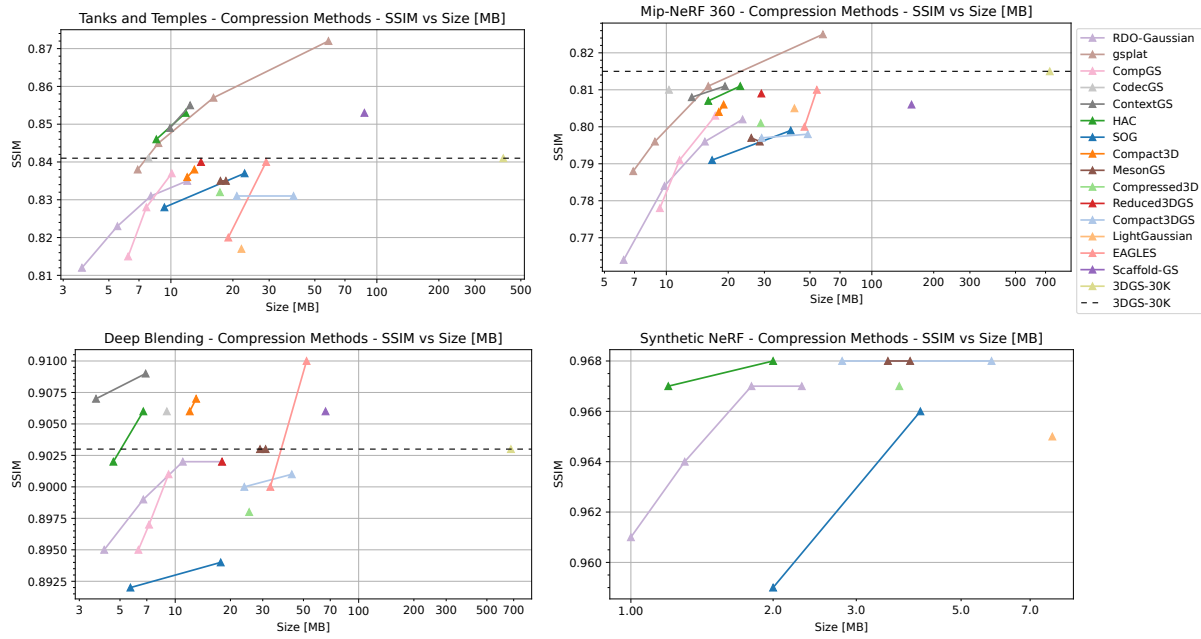


Figure 16: SSIM vs. Model Size (MB) for 3D Gaussian Splatting Compression Methods. The graphs compare different 3DGS compression methods across the Tanks and Temples, Mip-NeRF 360, Deep Blending, and Synthetic NeRF datasets. The x-axis represents the model size (in MB), while the y-axis represents the SSIM, indicating the visual quality.

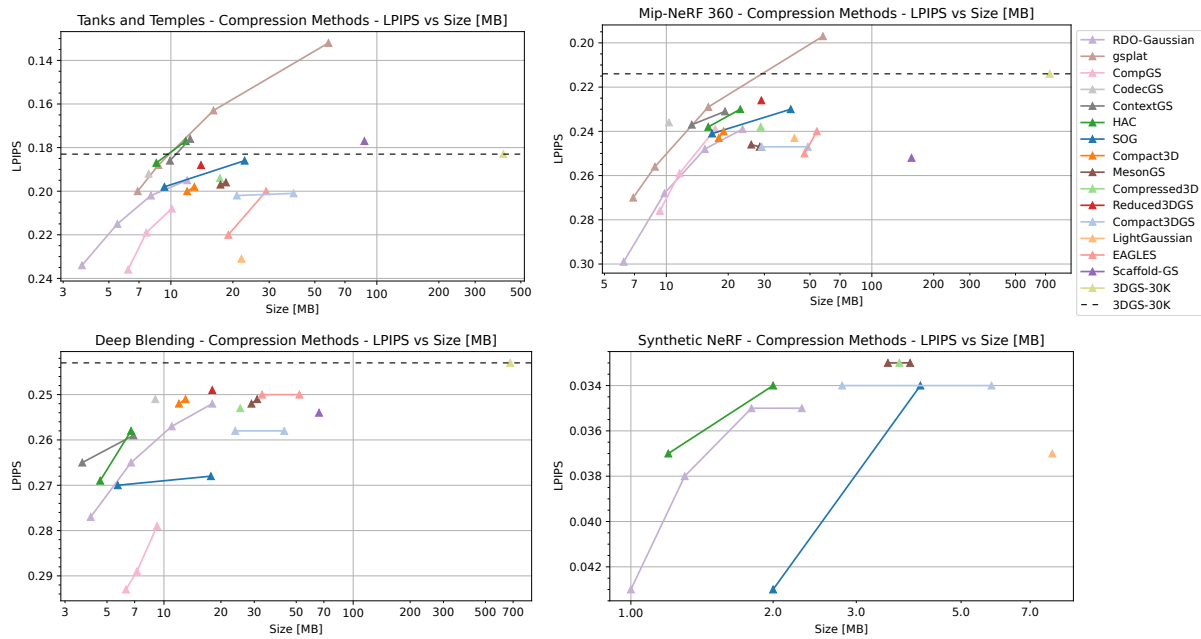


Figure 17: LPIPS vs. Model Size (MB) for 3D Gaussian Splatting Compression Methods. The graphs compare different 3DGS compression methods across the Tanks and Temples, Mip-NeRF 360, Deep Blending, and Synthetic NeRF datasets. The x-axis represents the model size (in MB), while the y-axis represents the LPIPS, indicating the visual quality.

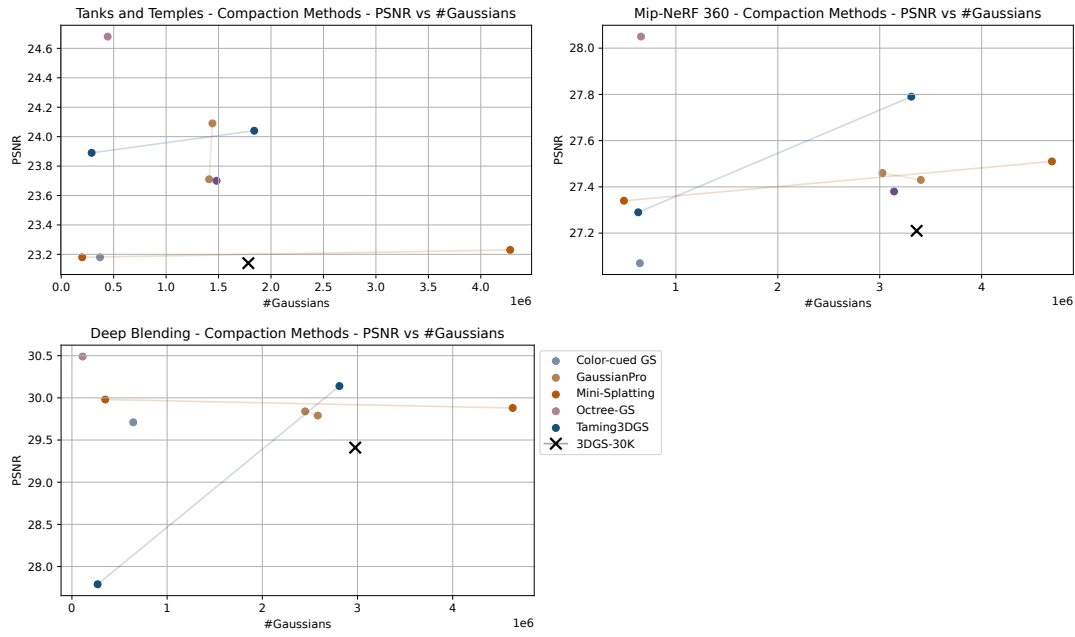


Figure 18: PSNR vs. $\#Gaussians \times 10^6$ for 3D Gaussian Splatting Compression Methods. The graphs compare different 3DGS compression methods across the Tanks and Temples, Mip-NeRF 360, Deep Blending, and Synthetic NeRF datasets. The x-axis represents the number of Gaussians $\times 10^6$, while the y-axis represents the PSNR, indicating the visual quality.

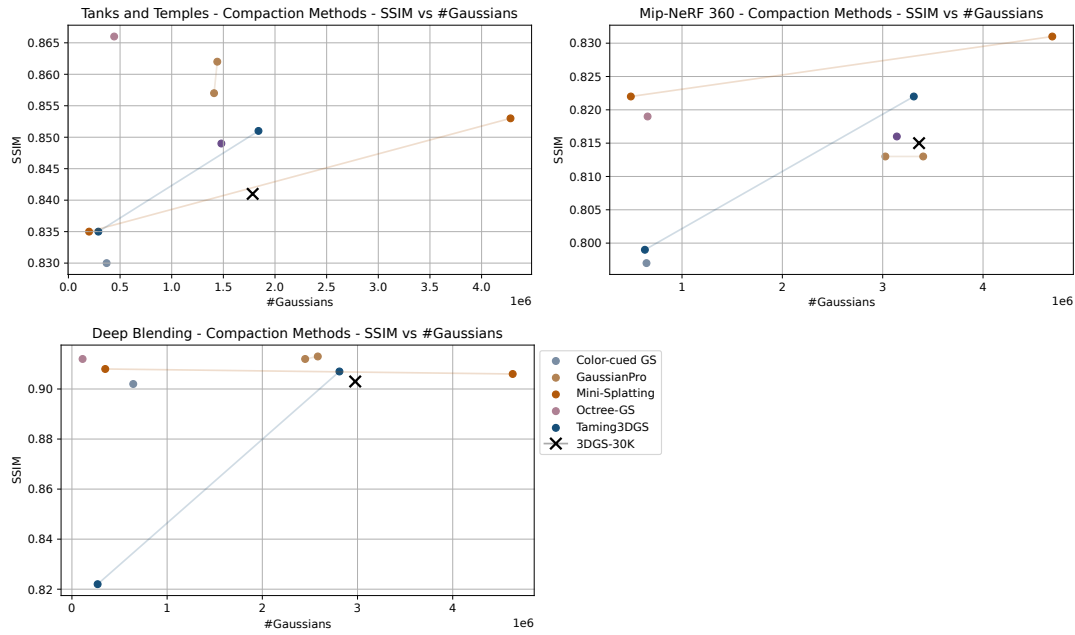


Figure 19: SSIM vs. $\#Gaussians \times 10^6$ for 3D Gaussian Splatting Compression Methods. The graphs compare different 3DGS compression methods across the Tanks and Temples, Mip-NeRF 360, Deep Blending, and Synthetic NeRF datasets. The x-axis represents the number of Gaussians $\times 10^6$, while the y-axis represents the SSIM, indicating the visual quality.

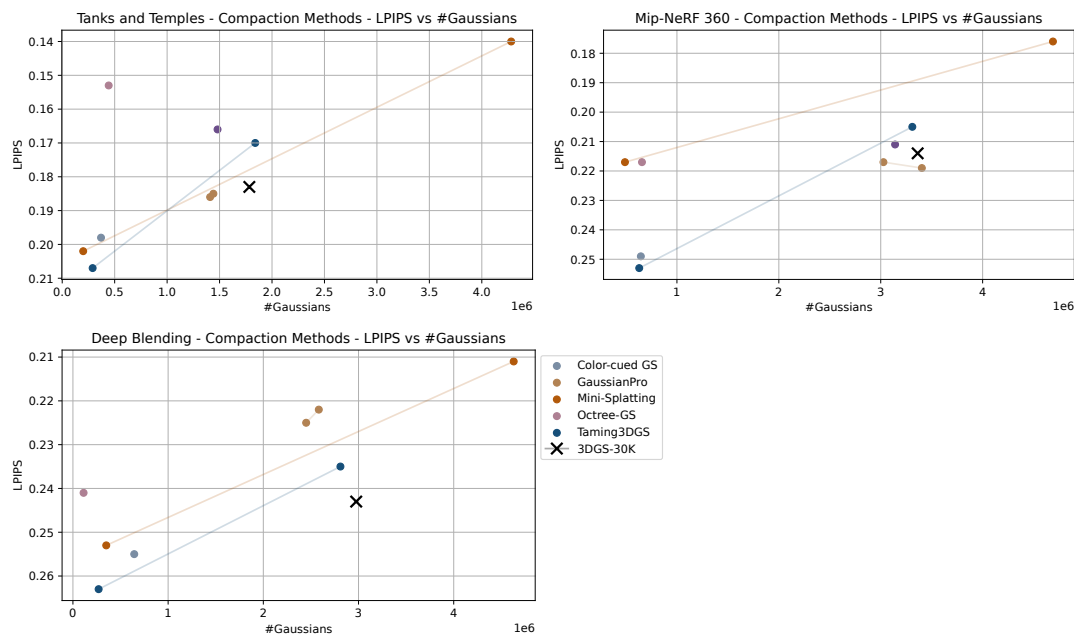


Figure 20: *LPIPS vs. #Gaussians $\times 10^6$ for 3D Gaussian Splatting Compression Methods.* The graphs compare different 3DGS compression methods across the Tanks and Temples, Mip-NeRF 360, Deep Blending, and Synthetic NeRF datasets. The x-axis represents the number of Gaussians $\times 10^6$, while the y-axis represents the LPIPS, indicating the visual quality.